

The Australian National University
2600 ACT | Canberra | Australia



Australian
National
University

School of Computing

College of Engineering, Computing
and Cybernetics (CECC)

Graph Model and Algorithms for Haplotype-resolved Assembly

— 24 pt Honours project (S1/S2 2023)

A thesis submitted for the degree
Bachelor of Advanced Computing (Research and Development) (Honours)

By:
Runpeng Luo

Supervisors:
A/Prof. Dr. Qing Wang (School of Computing)
A/Prof. Dr. Benjamin Schwessinger (School of Biology)
Prof. Dr. Lianrong Pu (Shandong University)

September 2026

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

September, Rumpeng Luo

Acknowledgements

First, I would like thank my supervisors, Dr. Qing Wang, Dr. Benjamin Schwessinger, and Dr. Lianrong Pu. Qing helps to administrate this Honours project. Benjamin and Lianrong provide valuable discussion through weekly meetings and guide me throughout the project. I deeply appreciate their effort and kindness, and impressed by their excellent research experience.

I sincerely thank Dr. Yu Lin, my first supervisor, advisor, and closest friend at ANU, who motivates the abstract idea of this project. He guided me to the area of Bioinformatics, and supervised my first research project. He is the most responsible, rigorous, patient, and kind-hearted person I have ever seen in my whole life. Although he is left, he will stay deeply in my heart, forever.

In addition, I would like show my sincere gratitude to my parents, who supports my undergraduate studies and provides emotional support when I felt stressed.

Finally, I would thank to NCI Gadi platform for providing the the services for running all benchmarks.

Abstract

Chromosome-level haplotype-resolved genome assembly is an important resource in molecular biology. Recent advancements in long-read sequencing technology have led to the emergence of near telomere-to-telomere assembly. Nevertheless, the majority of current assemblers tend to neglect the differences between haplotypes and construct a mixture of haplotype-specific sequences, which is not haplotype-resolved and leads to difficulty for downstream analysis.

This thesis focuses on solving the haplotype phasing problem, which aims to reconstruct chromosome-level haplotypes-resolved sequences. To begin, we propose and implement a novel haplotype phasing algorithm to phase the assembled contigs. We conduct comprehensive experiments on two real sequencing datasets of fungus to evaluate and benchmark our method against the state-of-the-art methods. The experimental results demonstrate that our proposed method outperformed all the baselines in effectiveness and efficiency.

The proposed phasing algorithms are publicly available at [GitHub](#).

Table of Contents

1	Introduction	1
2	Background	3
2.1	Basics about genome	3
2.1.1	Dikaryotic genomes	4
2.1.2	Terminologies	4
2.2	Genome sequencing	5
2.2.1	First-generation sequencing technology	6
2.2.2	Next-generation sequencing technology	6
2.2.3	Third-generation sequencing technology	6
2.2.4	Chromosome conformation capture methodology	6
2.3	Basics about Graph theory	7
2.3.1	Bridge of Königsberg problem	7
2.3.2	De Bruijn graph	8
2.3.3	Overlap Graph	10
2.4	The computational problems	11
2.4.1	Sequence alignment	11
2.4.2	Genome assembly	14
2.4.3	Haplotype phasing	18
3	Related Work	21
3.1	TGS Assemblers	21
3.1.1	Assemblers for error-prone TGS read	21
3.1.2	Assemblers for accurate TGS read	22
3.2	Haplotype phasing	23
3.2.1	HiC phasing module in Hifiasm	23
3.2.2	HiC phasing module in Verkko	24
3.2.3	GreenHill	25
3.3	Variant Calling	26
3.3.1	Kmer2SNP	26

Table of Contents

4	Methodology	29
4.1	Algorithm Overview	29
4.2	Genome Assembly and Variant Calling	29
4.2.1	Assembling LA and UL reads	29
4.2.2	Calling SNPs from LA reads	31
4.3	Heterozygous Contig Phasing	31
4.3.1	Identifying heterozygous contig pairs	32
4.3.2	Phasing heterozygous contig pairs	33
4.4	Homozygous Contig Phasing	36
4.4.1	Label propagation	36
4.4.2	Steiner Tree Formulation	39
5	Evaluation	43
5.1	Experimental Setup	43
5.1.1	Experimental dataset	43
5.1.2	Baselines and Evaluation Metrics	44
5.2	Experimental Results	45
5.2.1	Performance on Pt76 Datasets	45
5.2.2	Performance on Pst104E Datasets	48
5.2.3	Runtime, Memory and Implementation Details	54
6	Concluding Remarks	57
6.1	Conclusion	57
6.2	Future Work	57
A	Appendix: Chromosome-level Phasing results for Pt76 Dataset	59
B	Appendix: Chromosome-level Phasing results for Pst104E Dataset	67
C	Appendix: Command lines	75
	Bibliography	77

Introduction

A set of chromosome (or haplotype) carries genetic information inherited from a single parent. The variations between two set of chromosomes from a single individual can provide significant insights to biomedical research [Crawford and Nickerson \(2005\)](#) and disease analysis [Johnson et al. \(2001\)](#). Most diploid genome assemblers either collapse the variations between the haplotypes, which completely losing the variants, or leave the assembly result unphased, which cannot preserve long-range phasing information among multiple variants within the same haplotype.

With the advance of long read sequencing technology, long and accurate reads such as PacBio Hifi and Oxford Nanopore Duplex reads, ultra-long and error-prone reads such as Oxford Nanopore Simplex reads are developed and become popular for genome assembly. The state-of-the-art genome assemblers can utilize those advanced read technology and produce more accurate and complete assemblies. With the support of chromosome conformation capture (3C) data or trio (parental) data, the assemblies can be phased into two set of contigs, with each set representing a complete haplotype. *Haplotype-resolved* assembly is the process to generate phased assemblies from sequencing reads, while phasing the assemblies is referred to *haplotype phasing*.

Trio binning [Koren et al. \(2018\)](#) is developed for phasing haplotypes. However, parental sequencing data is not always available, such as wild animals or rust fungus. On the other hand, HiC data can be sequenced from a single sample without further constraint. It captures the long-range phasing information by having a HiC fragment forms a pair of reads that spans over the whole genome, which is one of the most popular auxiliary data applied to solve the haplotype phasing problem recently [Kronenberg et al. \(2021\)](#).

In this thesis, we focus on solving the haplotype phasing problem. First, we provide all the necessary background information and discuss about the related work. Then, we propose and implement a novel haplotype phasing algorithm to phase the assembled contigs using HiC data. We conduct comprehensive experiments on two real sequencing

1 Introduction

datasets about rust fungus. We evaluate and benchmark our method against the state-of-the-art phasing methods. Experimental results demonstrate that our proposed phasing algorithm outperformed all the baselines in terms of phasing accuracy and efficiency on the rust fungus datasets. Finally, we conclude the thesis with current limitations and future works.

Background

Before introducing the computational problem, we present some background for biology.

2.1 Basics about genome

Chromosome is made of protein and a DNA molecule, which carries the genome sequences that represents the genetic information. Genome sequence is represented as a list of nucleotide that carries the genetic information of an organism. There are four types of nucleotide (or base) in DNA: adenine (A), guanine (G), Cytosine (C), and thymine (T). Therefore, we can represent a chromosome as a string of letters A, G, C, and T.

A diploid genome is referred to two set of chromosomes. For example, human genome is diploid, with one set of chromosome inherited from father, and the other set from mother. Such set is called a haplotype. In addition, human genome (usually) consists of 22 pairs of autosomal chromosomes and one pair of sex chromosome. Except for diploid species, there are also haploid and polyploid species such as tomato (tetraploid), refer to Figure 2.1. This thesis focused on diploid species, in particular, rust fungi.

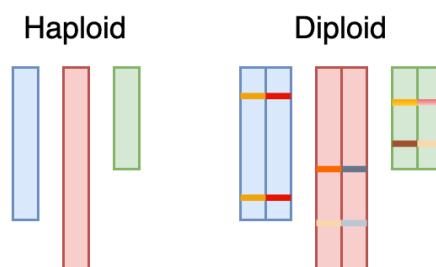


Figure 2.1: Haploid and diploid genome

2 Background

2.1.1 Dikaryotic genomes

Most animals and plants have multiple haplotypes coexisting within a single nucleus. In contrast, rust fungi possess a distinctive nuclear characteristic known as *dikaryosis*, wherein two nucleus coexist within the same cell, with each set of haploid chromosome (or haplotypes) residing in separate nuclei (refer to Figure 2.2 for a visualisation). This physical separation of haplotypes in different nuclei provides the potential for using HiC sequencing reads to separate and independently assemble them apart into haplotype level [Duan et al. \(2022\)](#).

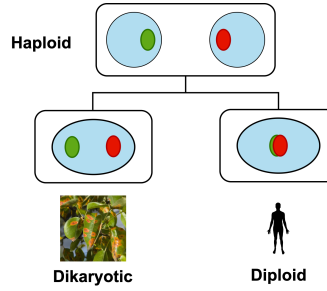


Figure 2.2: Dikaryotic genome. The blue ellipses refer to the cell. The green and red ellipses refer to distinct nucleus.

2.1.2 Terminologies

The *error rate* against a genome sequence is defined as the number of incorrect bases versus the total number of aligned bases, which is also referred to the estimated probability of the base call being wrong. The sequencing quality score (*q-score*) Q is defined as $Q = -10 \log_{10} e$, with e refers to the error rate.

Given a genome sequence S with $m = |S|$ bases, by mapping a set of sequenced reads onto S , the *sequencing depth* at i -th position is defined as the number of reads aligned to such position. The *coverage* of S is defined as percentage of positions that has depth at least 1 [Li et al. \(2009\)](#).

The *average depth* (or *sequencing coverage*) is computed as the sum of the sequencing depth across all bases, representing the number of times the whole genome be covered by the reads on average. Since the genome sequence is not always available before the assembly be done, we can also estimate the average depth or sequencing coverage by dividing the total read length by the estimated size of the genome.

In the Figure 2.3, we have a reference genome sequence with 60 bases. 5 reads are aligned to the sequence, with total read length of 162. The sequencing depth for position 24-30 is provided. The average depth is given as $162/60 = 2.7x$, where the coverage of the reference genome is 100%, since all the bases have nonzero sequencing depth.

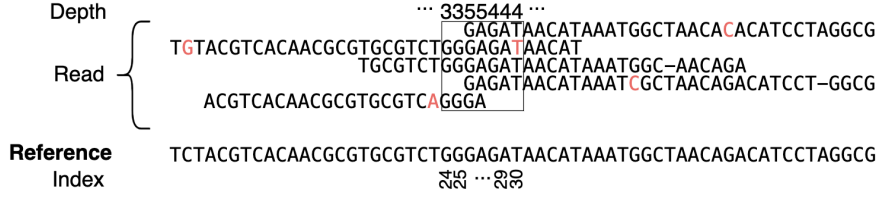


Figure 2.3: Sequencing depth and coverage. The red-coded symbol indicates mismatch, and ‘-’ indicates the deletion.

2.2 Genome sequencing

Whole genome sequencing is the procedure to determine the genomic sequence of an organism’s genome, it produces reads originating from the set of chromosomes in haploid level (or haplotype). Due to the technique limitation, current sequencing machines are unable to sequence the entire genome from the first nucleotide to last nucleotide, it can only sequence a fragment of a haplotype at a time. Based on that, genome sequencing normally starts with a sample of duplicated genomes, break the DNA into short fragments, and reads the exact genomic sequences from those fragments by sequencing machine. Since the reads are derived from both haplotypes of the entire genome, additional computations are required to attribute each read to its specific haplotype of origin, and further obtain the complete and separate haplotypes.

The DNA sequencing technologies have been developed and advanced for last four decades [Heather and Chain \(2016\)](#). Refer to Figure 2.4 for an illustration of genome sequencing on a diploid genome. In the following subsections, we will introduce the sequencing technologies and discuss their relatedness to the computational problems.

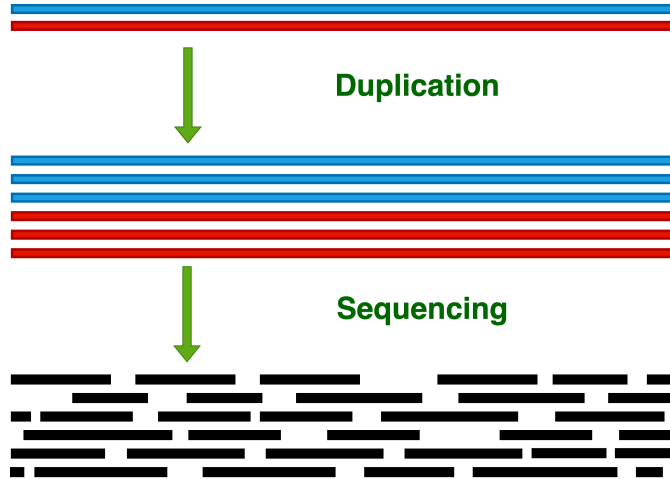


Figure 2.4: An illustration for sequencing diploid genome

2 Background

2.2.1 First-generation sequencing technology

Sanger sequencing is the First-Generation Sequencing (FGS) technology that introduced by Frederick Sanger around 1977 [Sanger and Coulson \(1975\)](#); [Sanger et al. \(1977\)](#), it can produce the reads with length about 400-900bp and accuracy up to 99.999%. However, the sanger sequencing technology suffers from low-throughput, expensive, and labor-intensive. Sanger sequencing was used by human genome project that costs roughly 3 billion US dollars, and took over ten years to finish the first human genome draft [Venter et al. \(2001\)](#).

2.2.2 Next-generation sequencing technology

Since 2005, the sanger sequencing has been replaced by the Next-Generation Sequencing (NGS) technologies [Bentley et al. \(2008\)](#); [Harris et al. \(2008\)](#). The majority of NGS technologies are dominated by Illumina company. NGS technology allow the entire genome to be sequenced at once (massively parallel) with high scalability, high through-put, and cost effective, thus revolutionize the sequencing and genomic studies. Nevertheless, there are disadvantages for NGS sequencing. First, NGS sequencing can produce short reads with length about hundreds of bases and roughly 0.1% error rate. Its error rate is higher than sanger sequencing (0.001%). Second, short reads cannot resolve long genomic repeats if the length of repeat sequences are longer than the read length. Also, short reads cannot span multiple variants along the genome and thus resulting in discontinuity of reconstructed haplotypes.

2.2.3 Third-generation sequencing technology

Due to the weakness of short reads on resolving repeat region, the third-generation sequencing (TGS) is developed to direct sequencing of single DNA molecule and obtain orders of magnitude longer reads compared to NGS Illumina short reads. The two most popular TGS technologies are Pacific Biosciences (PacBio)' high-fidelity (HiFi) reads [Eid et al. \(2009\)](#) and Oxford Nanopore (ONT) Simplex (or Duplex) reads [Deamer et al. \(2016\)](#), the former produces long (roughly 10k to 20k bp) and accurate (at least 99.9%) reads, where the latter produces ultra-long (50k to 150k bp) and error-prone (around 90% to 95%) reads. Duplex read is the latest Oxford Nanopore technology that can produce long reads with 99.9%-99.99% accuracy. Long reads have potential to resolve the genomic repeat region and span over multiple variants, which generates more contiguous genomic sequences and make chromosome-level haplotype assembly achievable. Figure 2.5 (a) and (b) describe the disadvantage and advantage of NGS and TGS read, respectively.

2.2.4 Chromosome conformation capture methodology

Chromosome conformation capture (3C) is a classical methodology to map chromatin interaction within regions of interest and across the genome [Lajoie et al. \(2015\)](#). There are various 3C-based method that originate from NGS sequencing further developed to

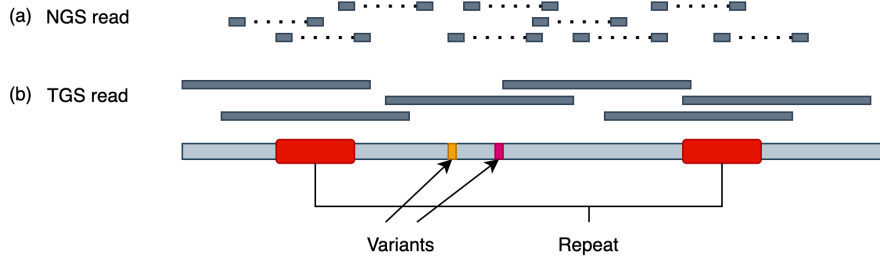


Figure 2.5: An example of genomic repeats and variants in a haploid genome. (a) NGS read cannot resolve genomic repeats and connect the variants (b) TGS read has potential to resolve the repeat and bypass multiple variants.

capture genomic interactions, such as 4C [Simonis et al. \(2006\)](#), 5C [Dostie et al. \(2006\)](#), and HiC [Lieberman-Aiden et al. \(2009\)](#). HiC uses unbiased deep sequencing to produce the set of reads. For instance, one HiC read is formulated as a pair of two short reads (with read length roughly 300bp each), which connected by the ligation junctions (or HiC link) of varying sizes.

In this thesis, we focus on TGS Oxford Nanopore (Simplex and Duplex) and HiC technologies. Table 2.1 summarises the main characters of those technologies.

Table 2.1: Summary of read technologies

	Read length	Error rate	Q-score
ONT Simplex Read	100kbp	10%	10
ONT Duplex Read	10kbp-30kbp	0.1%	30
Illumina HiC Read	(250×2) bp	0.1%	30

2.3 Basics about Graph theory

Graph plays important role in solving assembly problems. In the following subsections, we first describe a graph theory problem called “the Bridge of Königsberg problem” [Euler \(1956\)](#) that originally posed in 1736, which later inspires the development of De Bruijn graph. Then, we introduce two fundamental graph models, De Bruijn graph [De Bruijn \(1946\)](#); [Good \(1946\)](#); [Pevzner et al. \(2001\)](#) and overlap graph [Myers \(1995, 2005\)](#), and their associated concepts that are pivotal in tackling the genome assembly problem.

2.3.1 Bridge of Königsberg problem

Given the graph as Figure 2.6, the Bridge of Königsberg problem asks to devise a walk along the edges that visits every edge exactly once. This problem is equivalent to determine whether there exists an Euler path in the graph, which is solved by Euler theorem.

2 Background

Specifically, a graph has an Euler path if and only if it contains either 0 or 2 vertices with an odd degree. The concept of the Euler path, which entails traversing each edge exactly once in the graph, forms the foundational principle in the field of genome assembly [Compeau et al. \(2011\)](#).

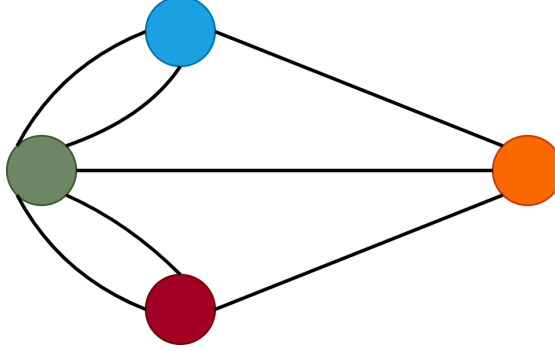


Figure 2.6: Bridge of Königsberg problem. An undirected graph with 4 vertices and 7 edges.

2.3.2 De Bruijn graph

A De Bruijn graph [De Bruijn \(1946\)](#); [Good \(1946\)](#) is a k -dimensional directed graph $G = (V, E)$ with n distinct symbols resides in $M = \{m_1, m_2, \dots, m_n\}$. Each vertex $v \in V$ carries a distinct sequence $s_v \in M^k$. The *prefix* of a sequence $s = (s_1, s_2, \dots, s_k) \in M^k$ is defined as the $(k-1)$ -tuple by removing the last symbol in s , *i.e.* $(s_1, s_2, \dots, s_{k-1})$. Similarly, the *suffix* of a sequence $s = (s_1, s_2, \dots, s_k) \in M^k$ is defined as the $(k-1)$ -tuple by removing the first symbol in s , *i.e.* $(s_2, \dots, s_k)$. Two vertices u and v are connected by an edge $e \in E$ if the suffix of s_u coincides with the prefix of s_v , and vice versa. Given n symbols and k dimensions, there are n^k possible sequences and leads to n^k vertices. Figure 2.7 shows an example of the De Bruijn graph with $k = 2$ and $M = \{0, 1\}$.

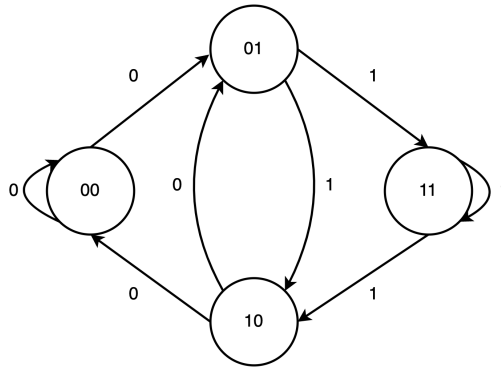


Figure 2.7: An example of De Bruijn graph with $k = 2$ and $M = \{0, 1\}$

Consider the superstring problem, which is to find a shortest circular superstring over

all possible substring with length k over a given alphabet M . For example, given $k = 3$ and $M = \{0, 1\}$, we have 2^3 possible distinct substrings $\{000, 001, 010, \dots\}$, the shortest circular superstring would be 0001110100, which contains all the substrings and be the shortest at the same time. The superstring problem was solved by Nicolaas de Bruijn via constructing a De Bruijn graph [De Bruijn \(1946\)](#).

First, we show that a De Bruijn graph is Eulerian. Given a De Bruijn graph $G = (V, E)$ with k dimension and n symbols, let v be a vertex in the graph with carried sequence s , there are n sequences that having same suffix coincide with the prefix of s , which results in in-degree n . Similarly, the out-degree is also n . According to Euler theorem, we've shown that a De Bruijn graph is Eulerian.

Second, we show that finding a Euler cycle in a De Bruijn graph solve the superstring problem. Given the superstring problem with k and M , one can construct a De Bruijn graph $G = (V, E)$ with $k - 1$ dimension and symbols M , any edge $e = (u, v) \in E$ also represents a substring with length k in alphabet M . Since V covers all possible $k - 1$ substrings ($|M|^{k-1}$), and we've shown that any vertex $v \in V$ has in-degree $|M|$ and out-degree $|M|$ from the first step, by the degree sum formula, there are $|M|^{k-1} \times |M| = |M|^k$ edges in the graph, which corresponds to $|M|^k$ all possible length k substrings. Hence, finding an Euler cycle and concatenate the first alphabet from consecutive vertices in the cycle would result in the shortest superstring.

De Bruijn graph (DBG) is the core concept to construct an assembly graph for NGS sequencing assemblers [Pevzner et al. \(2001\)](#); [Compeau et al. \(2011\)](#). For example, Given a fixed k , a group of NGS reads with read length l and error rate less than 1% (equivalently, 1bp error among 100bp) is sequenced from a haploid genome data with no repetitive region, we can chunk every read r into $l - k + 1$ k -tuples $\{r_1, r_2, \dots, r_{l-k+1}\}$, such that the suffix of r_i coincides with the prefix of r_{i+1} for $i \in [1, l - k]$. Such a k -tuple is often called a k -mer in the genome assembly problem. By extracting all distinct k -mers from all the reads, we can create a De Bruijn graph with vertices carrying the k -mers and $M = \{A, T, C, G\}$. Finding a Euler path (or circuit) from such graph would result in the underlying ground-truth genome in theory. Notice that the k -mer set extracted from the reads usually smaller than 4^k in practice.

In addition, modern DBG-based assemblers such as SPAdes [Bankevich et al. \(2012\)](#) could use a list of k -mer size instead of a fixed k value to construct the graph, which construct the DBGs with increasing k -mer size and merge the DBGs on the fly. It can use a larger k -mer size for a subgraph representing high repetitive genomic region, and a smaller k -mer size for a subgraph with low read coverage. Classic DBG assemblers are infeasible to handle error-prone long reads, since k -mers with large k always contain erroneous bases, while edges in DBG are formed only if $k - 1$ suffix-prefix perfect overlap exists. Thus, Long-range information cannot be preserved due to large amount of erroneous k -mers and lead to disconnection in the graph.

2 Background

2.3.3 Overlap Graph

Given a set of reads be sequenced from multiple copies of a haploid genome, the overlap graph [Myers \(1995, 2005\)](#) is defined by adding vertices for every read. An directed edge e is added from vertex u to v if the suffix of the read carried by u can be aligned to the prefix of the read carried by v , e is also referred to as *overlap* edge. The sequencing alignment is a well-studied computational problem and will be described at Subsection [2.4.1](#).

Consider there are three reads, namely r_1 , r_2 , and r_3 . Three overlap edges be added to the graph, (r_1, r_2) , (r_1, r_3) , and (r_2, r_3) . A transitive edge (r_1, r_3) carries redundant connectivity information and should be removed from the graph, which is part of the transitive edge removal step, a graph simplification strategy for overlap graph.

In contrast to the De Bruijn graph, the overlap graph can handle error-prone reads since perfect match was not required. However, the practical implementation of overlap graph suffers from the contained read problem. Given that edges in overlap graph carries suffix-prefix overlap information, a read contained by another longer read will be ignored during graph construction. This phenomenon can potentially introduce gaps (breaking connection) in the assembly graph. A more comprehensive discussion on the contained read problem is covered at Subsection [2.4.2](#).

Figure [2.8](#) describes an example of overlap graph and transitive edge removal step against a haploid genome [Li and Durbin \(2023\)](#).

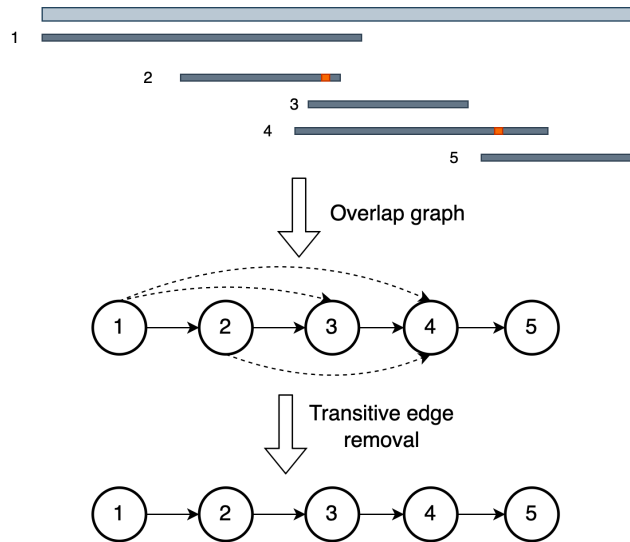


Figure 2.8: An example of overlap graph with 5 reads, the orange bars indicate the sequencing error on the read.

2.4 The computational problems

2.4.1 Sequence alignment

Sequence alignment is a fundamental problem that arises in the context of comparing two (pairwise) or among multiple genome sequences, a prerequisite for downstream analysis of genome similarity. In this thesis, we focus on pairwise sequence alignment since it is a key step to construct an overlap graph. In this subsection, we use R and Q to denote the reference and query genome sequences for alignment, respectively. We will explain two classical pairwise alignment algorithms based on dynamic programming, Needleman-Wunsch algorithm [Needleman and Wunsch \(1970\)](#) for global alignment and Smith-Waterman algorithm [Smith et al. \(1981\)](#) for local alignment. We further touch on the minimizer-based alignment algorithm, which is applied to the state-of-the-art long read sequence alignment software, Minimap [Li \(2016\)](#).

Suppose Q and R has similar length, which is normally occurred when the reference genome is coming from viruses or plasmids, which can be covered by a long read with read length roughly 10k-20kbp.

Global alignment is the way to align every bases in Q onto every bases in R that maximize the alignment score. There are three operations (illustrated in Figure 2.9) with corresponding alignment score be considered by Needleman-Wunsch algorithm, match (+1), mismatch (-1), and Indel (insertion or deletion) (-1). The scoring scheme is also applied to other alignment algorithms described later, unless specified otherwise.

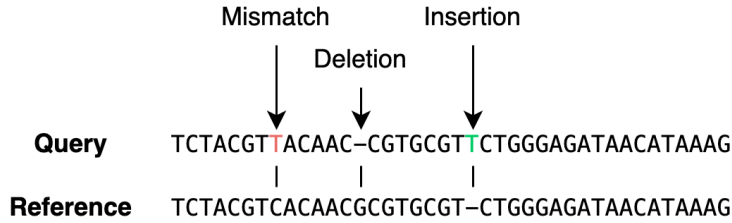


Figure 2.9: Illustration of alignment operations.

Needleman-Wunsch algorithm will first construct an empty 2D grid G with dimension $(|R| + 2) \times (|Q| + 2)$, with the top row carries the Q sequence (starting from $G[0][2]$) and the left-most column carries the R sequence (starting from $G[2][0]$). $G[1][1]$ is set to 0 as the base case, refers two the alignment score of 0 when aligning two empty sequence. $G[i][j]$ with $i \in [1, |R| + 1], j \in [1, |Q| + 1]$ refers to the alignment score for aligning $Q[0, i - 1]$ against $R[0, j - 1]$.

Consider the entries $G[1][j]$ with $j \in [2, |Q| + 1]$ correspond to align Q with increasing length against an empty reference sequence, which is a group of insertion operations. Similarly, the entries $G[i][1]$ with $i \in [2, |R| + 1]$ expresses a group of deletion operations by aligning an empty query sequence to increasing reference sequence.

2 Background

The inner entries $G[i][j]$ with $i \in [2, |R| + 1], j \in [2, |Q| + 1]$ can be computed from a recurrence relation to adjacent entries. $G[i][j]$ can be considered as alignment result from $G[i][j - 1]$ followed by an additional deletion operation on $R[j - 2]$. Similarly, $G[i][j]$ can also be considered as alignment result from $G[i - 1][j]$ followed by an additional insertion operation on $Q[i - 2]$. In addition, $G[i][j]$ can be considered as alignment result from $G[i - 1][j - 1]$ followed by an additional match (or mismatch) operation between $Q[i - 2]$ and $R[i - 2]$. The update on $G[i][j]$ can be covered by following recurrence.

$$G[i][j] = \max \begin{cases} G[i - 1][j - 1] \begin{cases} +1 & R[i - 2] = Q[j - 2], \\ -1 & \text{otherwise} \end{cases} \\ G[i - 1][j] - 1 \\ G[i][j - 1] - 1 \end{cases}$$

After filling the table, $G[|R| + 1][|Q| + 1]$ stores the global alignment scores, and by back-tracing the operations from $G[|R| + 1][|Q| + 1]$, the algorithm is able to obtain all the intermediate alignment operations.

An example for Needleman-Wunsch algorithm is provided in Figure 2.10. Global alignment traces are identified as red bars. Note that there are two alignment results with same alignment score, as follows.

CGTA-CGC
C-TAATGT

CGT-ACGC
C-TAATGT

Both alignments yield 1 insertion, 1 deletion, 2 mismatches, and 4 matches.

		Query							
Reference		" "	C	G	T	A	C	G	C
	" "	0	-1	-2	-3	-4	-5	-6	-7
	C	-1	1	0	-1	-2	-3	-4	-5
	T	-2	0	0	1	0	-1	-2	-3
	A	-3	-1	-1	0	2	1	0	-1
	A	-4	-2	-2	-1	1	0	-1	-1
	T	-5	-3	-3	-1	0	0	-1	-2
	G	-6	-4	-2	-2	-1	-1	1	1
	T	-7	-5	-3	-1	-2	-2	1	0

Figure 2.10: An example of Needleman-Wunsch algorithm

Smith-Waterman algorithm is a variation of Needleman-Wunsch algorithm and works for local alignment, which is specialized to determine the similar regions between two sequences instead of computing global similarity. The main difference of Smith-Waterman algorithm compare to Needleman-Wunsch algorithm is that no negative alignment score will be assigned to the entries of the grid. Back-trace is be done from the highest alignment score back to an 0 entry. By avoiding negative alignment score source from Indel and mismatch, Smith-Waterman algorithm can align Q to R with any length at any starting location.

An example for Smith-Waterman algorithm is provided in Figure 2.11. Local alignment traces are identified as red bars.

		Query							
Reference		" "	C	G	T	A	C	G	C
	" "	0	0	0	0	0	0	0	0
	C	0	1	0	0	0	1	0	1
	T	0	0	0	1	0	0	0	0
	A	0	0	0	0	2	1	0	0
	A	0	0	0	0	1	1	0	0
	T	0	0	0	1	0	0	0	0
	G	0	0	1	0	0	0	1	0
	T	0	0	0	2	1	0	0	0

Figure 2.11: An example of Smith-Waterman algorithm

The sequence alignment tools often align reads against a reference sequence, which is much longer than the reads. Smith-Waterman algorithm works for such local alignment purpose. However, both Needleman-Wunsch algorithm and Smith-Waterman algorithm suffers from the computation of the dynamic programming matrix in $O(mn)$ time and space complexity, which is infeasible when millions of sequences need to be aligned [Altschul et al. \(1997\)](#); [Kent \(2002\)](#). Modern alignment methods tend to use k -mer to search for sequence similarity combining with hash function. Given a hash function $\phi : \Sigma^k \rightarrow \mathbb{Z}$, k -mers are obtained from the reference and stored into a hash table. For every query sequence Q , the same hash function ϕ is applied to the extracted k -mers from Q , and looked up on pre-computed hash table for potential matches. Since most k -mers are overlapping, redundant information is stored in the hash table in this approach. To improve on this, Minimap [Li \(2016\)](#) chunk the sequence into a group of non-overlapping subsequences with length $w+k-1$ (also called *window*). Then, it selects the smallest k -mer in each window. The group of smallest k -mers in each window forms the (w, k) -minimizer, which serves as candidates and hashed into a hash table.

Figure 2.12 describes an example of $(5, 3)$ -minimizer over sequence TCTACGTCA-

2 Background

CAACGC, where the hash function $\phi(s) = (\phi(a_1)\phi(a_2)\phi(a_3))_4$ with $\phi(A) = 0, \phi(C) = 1, \phi(G) = 2, \phi(T) = 3$ is chosen to hash a 3-mer $s = a_1a_2a_3$. CAC is the minimal 3-mer at the second window according to the hash values.

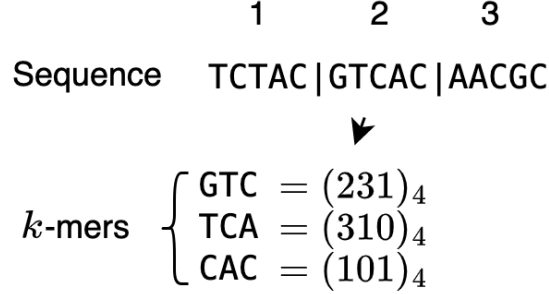


Figure 2.12: An example of single-strand (5, 3)-minimizer.

Based on the advantage of minimizer and hash functions, Minimap can perform sequence alignment with high accuracy and efficiency. Notice that the alignment result is sensitive to the main parameters such as the window size w and k -mer size.

2.4.2 Genome assembly

A human genome can be visualised as a book collection of 23 books, where each book has two analogous versions. Consider the book problem, you have a book with multiple identical copies, which all get sliced by a paper cutter, and result in a bunch of unordered variable-length paper pieces, the goal is to reconstruct the whole book by placing the paper piece into the correct spot.

Obviously, if you hold a copy of the original book (reference), you can trivially reconstruct the book by reading the copy and place the piece into correct spot. However, if the reference is unavailable, it is a hard computational problem to reconstruct the whole book.

The process to solve the book problem is similar to genome assembly. The key difference is instead of having only one book, we have a book collection with variable number of distinct books to recover. Instead of having one version per book, we normally have 2 (diploid) or more (polyploid) book versions get mixed up in the collection (sample). Thus, the genome assembly can also be visualized as a library problem.

In this thesis, we mainly consider the library problem with no reference, which is *De novo* assembly problem.

Figure 2.13 gives an overview of the De novo assembly pipeline. The problem takes the sequencing reads as inputs, and aims to output the full-length genome sequences (Chromosome-scale Haplotype resolved contigs) that spans the whole region or partial-length genome sequences (Haplotype-resolved contigs). The main computational prob-

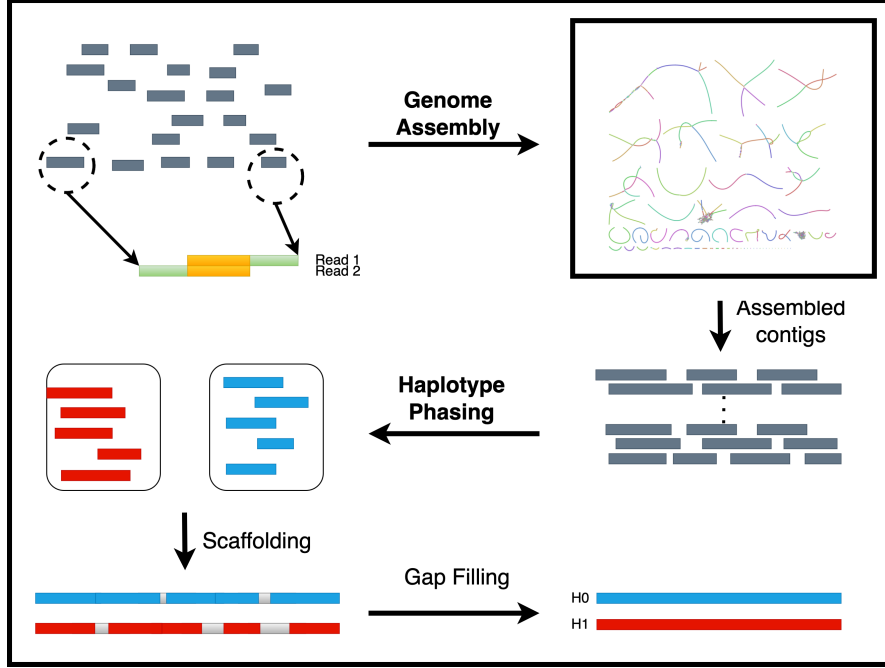


Figure 2.13: An overview of typical De novo assembly pipeline against diploid genome.

lem focuses on assembly graph construction and simplification, which is the core part of the genome assembler. Genome assembler may also performs read error correction by all-vs-all pairwise sequence alignment as a pre-processing step, as illustrated in Figure 2.14.

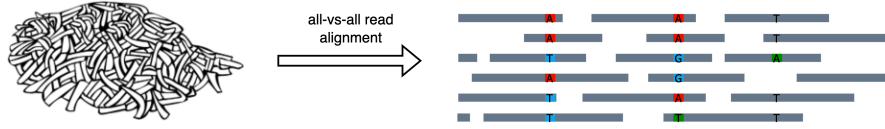


Figure 2.14: An example for read error correction prior diploid genome assembly. Red and blue bars indicate the variants on the two haplotypes, respectively. Green bars indicate the potential sequencing error. Gray bars indicate the matching regions through all-vs-all read alignment.

After error correction, the assembly graph is constructed based on either De Bruijn graph or overlap graph. For De Bruijn graph based assembler, k -mer counting is performed on reads to obtain a set of k -mers together with their multiplicities. The assembly graph can be defined as either node-centric or edge-centric [Li and Durbin \(2023\)](#). For node-centric approach, an assembly graph $DBG_v(k)$ is built with each vertex $v \in V(DBG_v(k))$ carries the k -mer, and each edge $e \in E(DBG_v(k))$ carries the $(k - 1)$ -mer overlap information, which obeys the definition of De Bruijn graph. Alternatively, edge-centric assembly graph $DBG_e(k)$ is built with each vertex $v \in V(DBG_e(k))$ carries the $(k - 1)$ -mer,

2 Background

where each edge $e \in E(DBG_e(k))$ carries the k -mer.

Notice that node-centric and edge-centric are inter-changeable, a node-centric graph $DBG_v(k)$ can be considered as an overlap graph with edges carrying $k - 1$ bp exact overlap. We obey the node-centric definition in this thesis. Since a small and fixed k value is infeasible to resolve the repeat genome region (leads to a complex loop), while a large k value may result in disconnection in graph for the low coverage genome regions. Therefore, multiplex DBG Bankevich et al. (2012, 2022) (mDBG) was introduced to merge multiple DBGs with varying k -mer size. It selects large k -mer size to resolve the repetitive genome region and small k -mer size to keep the connectivity of the graph on low coverage regions. An example of mDBG is given in Figure 2.15, notice that either $DBG_v(5)$ or $DBG_v(6)$ results in tangled graph, while the $mDBG_v(5, 6)$ can produce a graph without complex loop and fragmentation.

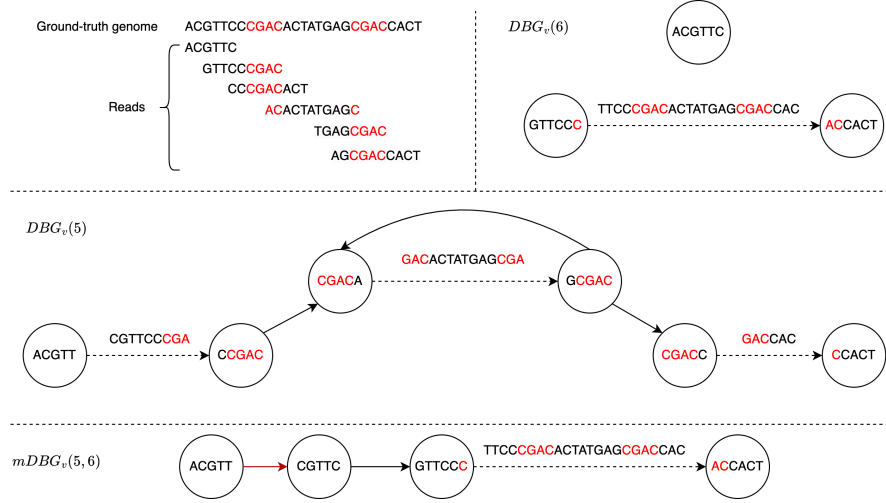


Figure 2.15: An example on multiplex De Bruijn graph construction using $k = 5, 6$. The dotted line (and carried sequence on the line) simplifies the De Bruijn graph representation, The dashed line is the exact De Bruijn graph edge. CGAC is a genomic repeat and colored red for illustration.

Furthermore, notice that the storage of all k -mers from the reads would result in a considerable memory overhead. To mitigate this issue, a minimizer-based sparsification Ekim et al. (2021); Rautiainen and Marschall (2021) technique is introduced, such that only minimizers (refer to Subsection 2.4.1) are kept in memory. This technique strongly reduces the memory consumption and speeds up the assembly graph construction. The combination of multiplex DBG and minimizer builds the foundation of the modern DBG-based assemblers to assemble NGS or accurate TGS reads.

For overlap graph based assembler, all-vs-all pairwise sequence alignment is done, each vertex represents a read, and a directed edge is added if the suffix of a read A can be aligned to the prefix of another read B . If the overlap graph is permitted to have inexact

suffix-prefix alignment, it may result in a more contiguous graph, but ignores the rare mutations in repetitive region such as tandem duplication and single nucleotide polymorphism (SNP). Instead, if the overlap edge disallows inexact matches, two reads form an directed edge only if the suffix-prefix alignment is exact without mismatches. This strategy is supported by highly accurate TGS technologies appeared recently such as PacBio Hifi and Oxford Nanopore Duplex reads. Compared to the DBG-based method, an overlap graph can preserve long-range information since reads are much longer than k -mer, which has potential to resolve repeat region more accurately if the reads are longer than the repeat. However, overlap graph suffers from the contained read problem [Cheng et al. \(2023\)](#). A short read completely contained in a longer read is intuitively discarded by the assemblers since it seems to provide redundant information. However, such removal may result in disconnections (or gaps) in the assembly graph. An example is given in [Figure 2.16 Li and Durbin \(2023\)](#).

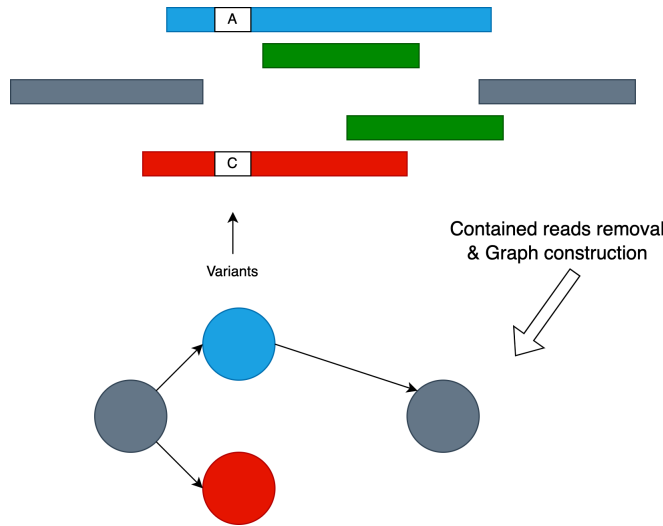


Figure 2.16: An illustration of contained read problem in overlap graph construction.

After the assembly graph is constructed, various heuristics such as tip (vertices with degree at most 1) removal and simple path contraction (merge a non-branching path to a single vertex and remain connected to the graph) are employed to simplify the graph. Bubbles are left on the graph because they are normally caused by variants between haplotypes and cannot be resolved by reads. An example is given in [Figure 2.17](#).

Finally, the vertices are extracted from the assembly graph and output as the contigs that either specific to a single ground-truth haplotype, or homozygous to both haplotypes. Those contigs will then be phased into two bins that corresponds to two haplotypes. Ultimately, the full-length haplotypes can be obtained by scaffolding the contigs (produce ordered sequence of contigs) and filling the gap between the adjacent contigs in the scaffolds.

2 Background

2.4.3 Haplotype phasing

After the assembly graph is constructed and simplified, a chain of bubbles is left on the graph, which corresponds to specific haplotypes (as illustrated in Figure 2.17). It is infeasible to perform path traversal from source to sink in the assembly graph with bubbles, since there are no information be provided to support the traverse across the bubbles. Most assemblers tend to collapse the bubbles into a mosaic consensus sequences, which does not correspond to any of the ground-truth haplotypes. On the other hand, the fully-resolved haplotypes can be applied to determine the parental origin of alleles and to test for association with disease [Browning and Browning \(2011\)](#). Haplotype phasing problem aims to resolve the chain of bubbles and recover the haplotypes separately.

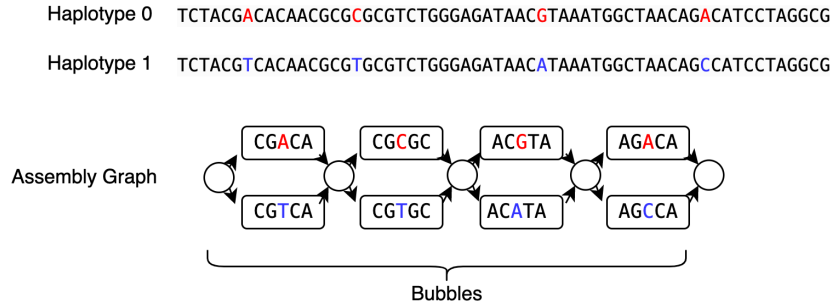


Figure 2.17: An illustration of bubbles in a simplified assembly graph. Haplotype 0 and 1 refers to the ground-truth haplotype we aim to construct.

If the reference sequence is available in advance, one can align the reads onto the reference, identify the variants, bridge the gaps between adjacent variants, and finally resolve the two haplotypes. Such reference-based phasing strategy can produce accurate haplotypes, but relies on the existence of high quality reference sequence, which is infeasible for novel species. In addition, it may not be able to detect variations on samples coming from species with high mutation rates.

Another approach is reference-free, which aims to recover the two set of haplotypes without reference, such method requires additional information such as 3C reads (described at Subsection 2.2.4) or NGS reads (refers to trio data) coming from both parents. 3C reads such as HiC can span over the whole genome and produce long-range phasing information, which links two loci if they are spatially close in a nucleus [Selvaraj et al. \(2013\)](#).

Nevertheless, 3C reads may occasionally produce linkage information across different haplotypes and introduce noises to the phasing [Cheng et al. \(2022\)](#). Alternatively, trio data is obtained by sampling and sequencing NGS reads from the genome of both parents, reads sequenced from the offspring can be binned to two parental groups separately, and further assembled into two haplotypes independently [Koren et al. \(2018\)](#). Compared to 3C reads, trio data is more reliable for haplotype phasing. However, parental data may

2.4 *The computational problems*

not always available due to lack of parents, such as wild animals and rust fungus.

In this thesis, we focus on incorporating the HiC reads to produce better phasing results under reference-free approach with high phasing accuracy.

Related Work

In this chapter, we start with describing the state-of-the-art TGS assemblers, followed by the existing reference-free phasing approaches that be used for benchmark. Various third-party tools be employed by our method are also covered.

3.1 TGS Assemblers

TGS assemblers assemble (error-prone or accurate) TGS reads by constructing assembly graphs and output associated contigs from the graph.

3.1.1 Assemblers for error-prone TGS read

Flye [Lin et al. \(2016\)](#); [Kolmogorov et al. \(2019\)](#) and Canu [Koren et al. \(2017\)](#) are two assemblers that robust to handle error-prone TGS reads, they are developed at an early stage of TGS technology and leads to innovative ideas on processing long reads with high error rates.

Canu performs genome assembly in a pipeline with three stages. First, Canu corrects the sequencing errors from the reads by all-vs-all pairwise sequence alignment (refer to Subsection [2.4.2](#) for an example of read error correction), it uses 2-level (global and local) filtering to handle the variation among repeat regions. Then, Canu trims the corrected reads if it lacks of overlap support from other reads. At the last step, Canu constructs the assembly graph based on the concept of best overlap graph [Miller et al. \(2008\)](#) (similar concept is described at Subsection [2.3.3](#)), extracts vertices from the pruned assembly graph, and results in a set of contigs (sequence fragment carried by the vertices) as final output.

The output contigs are either haplotype-specific or homozygous to both haplotypes. The read error correction step from Canu can significantly shrink the error rate of the input

3 Related Work

error-prone TGS reads, which makes the assembly graph construction more accurate. However, it takes quadratic time complexity to perform the error correction over a large amount of raw reads due to all-vs-all read alignment, which is highly time-consuming.

Differ to Canu, Flye does not perform read correction. Instead, it first generate a set of *disjointigs* by concatenating the error-prone raw reads with consideration of overlaps. Then, it concatenates all the disjointigs into a single string in an arbitrary order and construct an accurate assembly graph based on repeat graph, which characterizes the structure of repetitive genome region. Flye proves that the repeat graph constructed by concatenating all disjointigs in an arbitrary order leads to the correct representation of the assembled ground-truth genome. After that, Flye resolves the repeats by considering the number of reads passing in to (and out of) and coverage ratio to adjacent vertices. Finally, Flye simplifies the assembly graph via various graph pruning techniques and outputs contigs which corresponds to non-branching paths in the graph.

Flye in general output genome sequences that be served as a mosaic consensus by collapsing haplotype-specific vertices in assembly graph, which does not correspond to any specific haplotypes and result in numerous switching errors among heterozygous regions.

3.1.2 Assemblers for accurate TGS read

Hifiasm [Cheng et al. \(2021\)](#) is designed to handle TGS read with error rate less than 0.1%, specific to PacBio HiFi reads. Similar to Canu, Hifiasm starts to perform all-versus-all read overlap and read error correction. Then, it constructs a string graph with vertices as reads, and edges as end-to-end overlap. Thanks to the extremely low error rate on corrected reads, Hifiasm only adds an edge if they have near perfect overlap, which greatly simplifies the assembly graph. After that, Hifiasm can optionally utilize the error-prone ultra-long reads (10% error rate) to bridge the bubbles and fill the gaps from low coverage regions. For instance, Hifiasm aligns the ultra-long reads [Cheng et al. \(2023\)](#) against the vertices (HiFi reads). For each ultra-long read u_i , it may aligns with a sequence of vertices $(h_0, h_1, \dots)$, so called *Ultra-long integer sequence*. Given the set of ultra-long integer sequences, Hifiasm constructs a ultra-long integer graph, which is an undirected graph with ultra-long reads as vertices, and edge is added between two vertices if they have end-to-end perfect match from their corresponding ultra-long integer sequences. Finally, Hifiasm merges both HiFi assembly graph and ultra-long integer graph, and resolves the bubble regions if applicable. Finally, if either Trio data or HiC data is provided, Hifiasm can phase the assembly result into two disjoint sets of contigs, one for each haplotype. The HiC phasing method for Hifiasm will be discussed at Subsection [3.2.1](#).

Verkko [Rautiainen et al. \(2023\)](#) is the other TGS assembler that takes both accurate long (LA) read and error-prone ultra-long (UL) read as inputs, and generates haplotype-resolved assemblies. First, Verkko employs Canu [Nurk et al. \(2020\)](#) to perform error correction and homopolymer sequence compression to mitigate the homopolymer insertions and deletions error in long read sequencing. Then, Verkko utilizes MBG [Rautiainen and](#)

Marschall (2021) to construct a minimizer-based sparse de Bruijn graph (LA graph) with a predefined k -mer size ($k = 1001$ by default) using accurate long read only. After that, Verkko uses a sparse Smith-Waterman algorithm (refer to Subsection 2.4.1) to align UL reads against the vertices in the graph. Verkko fills gaps, resolves repeats (circuit structure), and untangles the bubbles (homozygous region) based on UL alignments. For instance, Verkko attempts to connect two vertices in the graph if they are supported by (at least 3) UL alignments, and resolves the bubble region by applying the multiplex de Bruijn graph algorithm Bankevich et al. (2022).

Once the graph is cleaned and simplified, which is referred to ULA graph, Verkko incorporates the auxiliary data (either Trio or HiC data) to phase the contigs into two bins representing the parental and maternal haplotype. The HiC phasing method in Verkko is discussed at Subsection 3.2.2.

In summary, Canu and Flye are two TGS assemblers capable to handle error-prone (error rate up to 15%) long reads, they proposed innovative ideas to resolve repeat structure when lack of ultra-long read, such as repeat graph construction. When the TGS reads get much longer and accurate, Hifiasm and Verkko are the only two assemblers that can utilize both accurate long read (error rate less than 0.1%) and ultra-long reads to bridge more bubbles and repeat region, which leads to more accurate and simplified assemblies.

3.2 Haplotype phasing

Haplotype phasing aims to separate the haplotype-specific contigs that belongs to two haplotypes post assembly. Current phasing methods utilize either Trio or HiC data for phasing. In this thesis, we focus on HiC data since parental data is not always available.

In this section, we discuss the state-of-the-art phasing algorithms proposed by Hifiasm Cheng et al. (2022), Verkko Rautiainen et al. (2023), and GreenHill Ouchi et al. (2023), which will be our main benchmark.

3.2.1 HiC phasing module in Hifiasm

After the assembly graph construction, Hifiasm maps HiC segments to the assembly graph, with both ends of the HiC segment are mapped to two different heterozygous positions. Since HiC read is error-prone and short ($300 \times 2\text{bp}$), where general purpose read mapper such as BWA-MEM Li (2013) and Bowtie2 Langmead and Salzberg (2012) tends to maximize the alignment sensitivity and accuracy against accurate NGS read, which is time consuming to map error-prone HiC data with large coverage. Therefore, Hifiasm indexes the unique 31-mers from the vertices (unitigs) of the assembly graph, a HiC segment is mapped if it matches two non-overlapping indexed 31-mers from each end, respectively. Such k -mer-based alignment method can efficiently map noisy HiC data and maintain comparable accuracy versus general purpose read mapper. A HiC segment mapping is *cis* if it maps to two heterozygous unitigs that belongs to same haplotype, or

3 Related Work

trans otherwise. Hifiasm attempts to model the probability of *cis*-mapping and *trans*-mapping after HiC mapping. Since the probability is related to the insert size [Edge et al. \(2017\)](#), Hifiasm employs an empirical multiple round method to fit the probability function, and determines the *cis*- and *trans*-mappings. Finally, Hifiasm formulates the phasing problem as a max-cut problem, and uses a stochastic algorithm to solve for near-optimal solution.

Although Hifiasm can efficiently align HiC reads and solve the phasing problem using a max-cut formulation, it assumes that the bubble region normally coming from the heterozygous region, where the non-bubble region are coming from homozygous region, and phase them into two bins separately. However, it is common that the two heterozygous contigs belong to the same position of two haplotypes within the same chromosome do not form a bubble structure in the assembly graph, and such phasing method may only be able to capture one side of the heterozygous contig, leads to gaps in final phased assemblies.

3.2.2 HiC phasing module in Verkko

Once the ULA graph be constructed, Verkko¹ uses general purpose short read mapper BWA-MEM [Li \(2013\)](#) to map the HiC read segments against the contigs from the ULA graph, and obtain a set of distinct contig pairs supported by alignment from each end of HiC segments.

Then, Verkko performs a self alignment for the contigs, and aims to find a set of matching pairs exceeding the similarity threshold (the threshold is determined by haplotype divergence, max allowed is 0.2). Finally, Verkko employs a greedy bisection algorithm to bipartite all the located contig pairs, which results into two bins corresponding to two independent haplotypes, respectively.

From the above pipeline, Verkko may not be able to handle the mutations on tandem duplication, which is two or more sub-sequences in a haplotype that is duplicated to each other but having variation on a small portion of nucleotide, an example is given in [Figure 3.1](#).

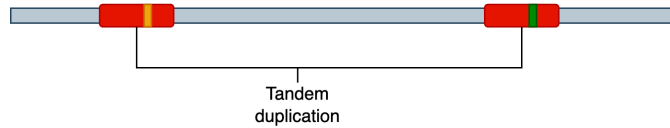


Figure 3.1: An illustration of tandem duplication.

According to the self alignment step, tandem duplication may be mis-classified as matching pairs due to high similarity with low variations, which leads to switch errors during

¹The Verkko HiC phasing method is discussed based on the analysis through their <https://github.com/marbl/verkko/blob/master/src/scripts/hicverkko.py>, which is not covered by their manuscripts

later bisection step.

3.2.3 GreenHill

GreenHill [Ouchi et al. \(2023\)](#) is a post-assembly tool that performs both scaffolding and phasing for diploid genome using the assembled contigs, long reads, and HiC reads, it is the state-of-the-art scaffolding and phasing method that be developed recently. We focuses on the haplotype-ignorant styles inputs, since GreenHill varies the algorithm based on the types of input it takes. First, it performs a self-alignment for assembled contigs, and associates a contig with its opposite contig, where the former has the longest alignment to the latter, and vice versa. Various heuristics are applied at this stage to ensure that a contig can only be paired with at most one opposite contig, and minimize the chances of pairing homozygous contigs. Then, GreenHill maps the long read (same input to the assembly) against the contigs using Minimap2 [Li \(2018\)](#), it selects the mapping according to the sequence identity, which is defined as the ratio of number of matches versus the total alignment length. HiC reads are also be mapped using k -mer exact match (similar to Hifiasm HiC phasing method). After that, GreenHill performs consensus scaffolding using both long reads and HiC reads, where HiC reads are also be applied to perform error-correction based on alignment result from previous step. Scaffolding is another computational problem that aims to group and order the assembled contigs, the details of scaffolding algorithm is omitted since it is not our main focus in this thesis. Once the scaffolding is done, GreenHill uses HiC reads to phase the contigs and classifies them into two bins, which corresponds to two independent haplotypes, respectively.

For instance, an array of blocks $A = (B_1, B_2, \dots, B_n)$ are produced after scaffolding, where each block B_i consists of either 1 (identified homozygous contig) or 2 (identified heterozygous contig pair) contigs. All the first contig in heterozygous blocks are considered to be in haplotype 0, and all the second contig are considered to be in haplotype 1. The homozygous contigs are considered to be both haplotype 0 and 1. Then, it assigns supporting linkage information based on HiC or long read alignment between any two distinct contigs from the array. Once the linkage is added to the array, GreenHill connects a pair of distinct contig blocks from left to right by considering the largest supporting links. Given the selected pair of contig blocks $B_i = (c_0^i, c_1^i), B_j = (c_0^j, c_1^j), i < j$, without loss of generality, if there are more links connecting c_0^i to c_1^j rather than c_0^j , GreenHill will swap the contigs in B_j to maximize the connection stays within the haplotypes. GreenHill iteratively connects all contig blocks if there are sufficient supporting links, until no pairs of blocks can be connected. Blocks are still disconnected afterwards will be randomly assigned to each haplotype bin. Refer to the Figure S11 of supplementary material of GreenHill paper for the detailed description.

GreenHill produces comparable phasing result versus Hifiasm (HiC) and Verkko (HiC) mode, while also performs scaffolding at the same time. However, there are potential weaknesses for this algorithm. First, since the contig pairs are identified by performing

3 Related Work

self-alignment on contigs, GreenHill may also pairs two contigs that corresponds to tandem duplication region, which results to switch errors. Second, since GreenHill randomly determine the phase of the unconnected blocks, it may introduces more switch errors, those unconnected blocks may be assigned more confidently by utilising the topological structure of the assembly graph.

3.3 Variant Calling

Recall the diploid genome consists of two haplotypes, *heterozygous variant* refers to the differences between two haplotypes within the same chromosome. Single nucleotide polymorphism (SNP) is the most common heterozygous variant, which is defined as different nucleotide at the same position on at least two different haplotypes. Since SNP has multiple analogous definitions, in the thesis, we do not consider the occurrence percentage of such variant within a population but rather focuses on a single individual. An example of SNP is given in Figure 3.2.



Figure 3.2: The definition of single nucleotide polymorphism (SNP).

3.3.1 Kmer2SNP

Kmer2SNP [Li et al. \(2020\)](#) is a reference-free variant calling tools that directly calls SNPs from accurate sequencing reads, such as NGS short read or TGS accurate long read. It takes the reads, k -mer size, and estimated homozygous sequencing coverage (refer to Subsection 2.1.2) as inputs, and outputs a group of k -mer pairs that carrying the SNPs.

First, Kmer2SNP performs k -mer counting using DSK [Rizk et al. \(2013\)](#), which counts the frequency of the distinct k -mers from the raw reads, and produces a histogram that carries the frequency information. Then, findGSE [Sun et al. \(2018\)](#) is utilized to discover the heterozygous peak of the histogram based on the homozygous sequencing coverage threshold. Kmer2SNP observes that the frequency of heterozygous k -mers are normally halve of the frequency of homozygous k -mers, since the former appears in one haplotype only, but the latter appears in both haplotypes. After the heterozygous k -mers be obtained, any two k -mers are paired if they differ at the $(\frac{k+1}{2})$ -th position only, weight is assigned to k -mer pairs based on the support from matching adjacent overlapping k -mers on both side. Finally, k -mer pairs are captured by performing a maximum weighted matching against the k -mer pairs and the assigned weight.

Kmer2SNP can effectively extract most true-positive SNPs, while maintaining low false-positive rate. It is employed by our proposed phasing algorithm to perform variant

calling as a preprocessing step.

Methodology

In this chapter, we introduce a novel reference-free haplotype phasing algorithm that serves as a post-assembly method for the state-of-the-art assemblers. It utilizes the HiC data to accurately phase the assembled contigs into two bins that corresponds to two independent haplotypes.

4.1 Algorithm Overview

The algorithm takes accurate long (LA) reads (such as PacBio HiFi reads or Oxford Nanopore Duplex reads), error-prone ultra long (UL) reads, HiC reads, and homozygous sequencing coverage threshold (refer to Subsection 3.3) as the inputs, and outputs a phased haplotype-resolved assemblies. First, the algorithm employs general purpose assemblers that supports both LA reads and UL reads to construct the initial unphased assemblies, which consists a set of assembled contigs and an assembly graph. Then, the algorithm extracts the heterozygous SNP k -mer pairs using Kmer2SNP, and phases the detected heterozygous contigs by solving the maximum weighted matching and graph bisection problem using both SNP k -mer pairs and HiC data. Finally, if assembly graph is explicitly provided by the assembler, the algorithm can utilize the assembly graph to further phase the undetected heterozygous and homozygous contigs by solving the Steiner tree problem. Refer to Figure 4.1 for an overview of our algorithm. Details of each step are explained in the following sections.

4.2 Genome Assembly and Variant Calling

4.2.1 Assembling LA and UL reads

Given the LA reads and UL reads, haplotype-resolved assembler is employed to utilize both data types and produce a set of unphased assembled contigs, together with an

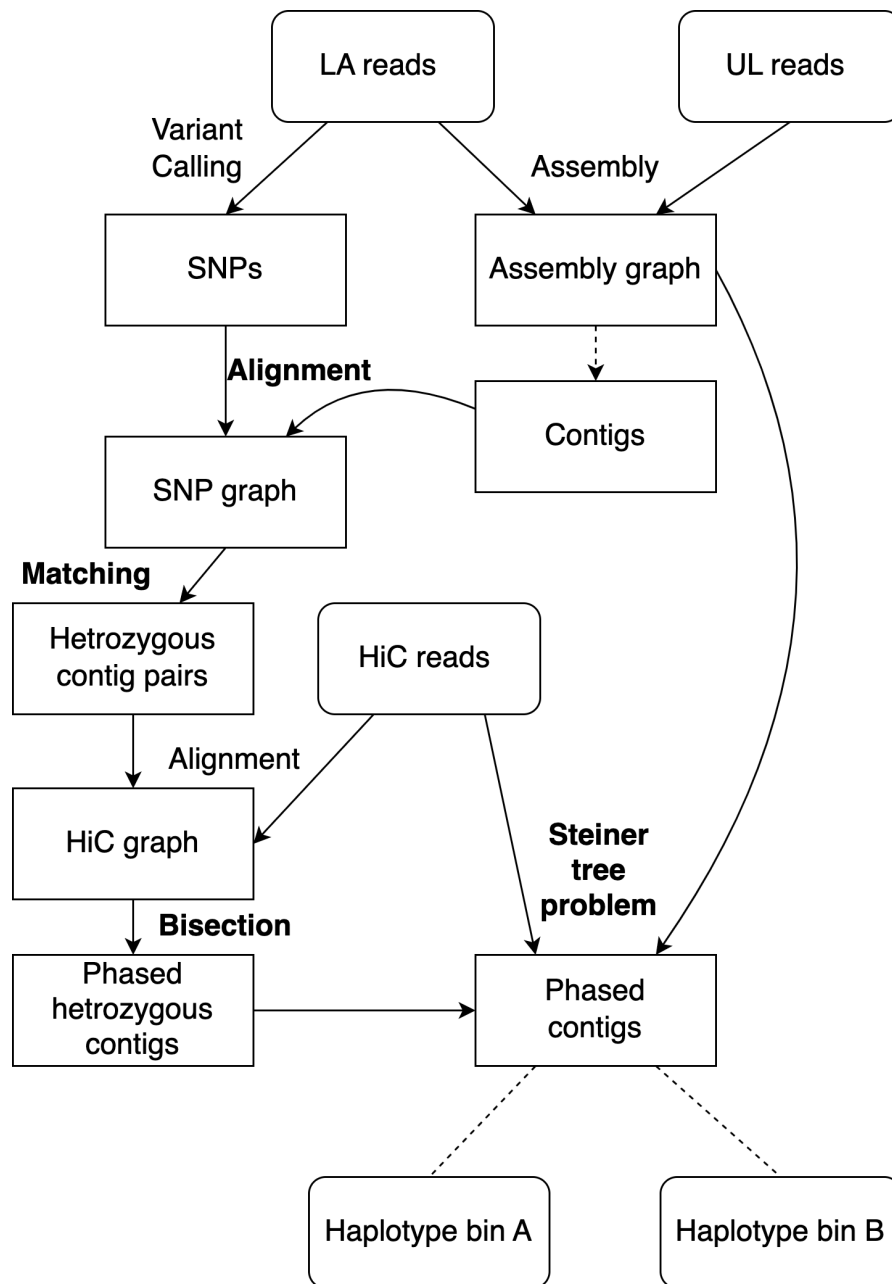


Figure 4.1: The overview of proposed phasing algorithm

assembly graph.

4.2.2 Calling SNPs from LA reads

The algorithm utilizes Kmer2SNP to call SNPs directly from LA reads, since k -mer size is relevant to the heterozygosity rate of the genome, it is chosen based on the selection criteria from the paper [Li et al. \(2020\)](#). Homozygous sequencing coverage is referred to the total sequenced bases divided by the diploid genome size, it is provided to Kmer2SNP as the required argument as well. Note that although the reference genome is unavailable before the assembly, genome size can still be estimated by either comparing against similar species, or performing a haplotype-ignored pre-assembly and employing the total contig lengths as the genome size.

A SNP is defined as a k -mer pair $P_i = \{p_0^i, p_1^i\}$, all k -mers that are adjacently overlapped to p_0^i and p_1^i will also be considered if their hamming distance is 1. Figure 4.2 gives an example of a SNP k -mer pair.

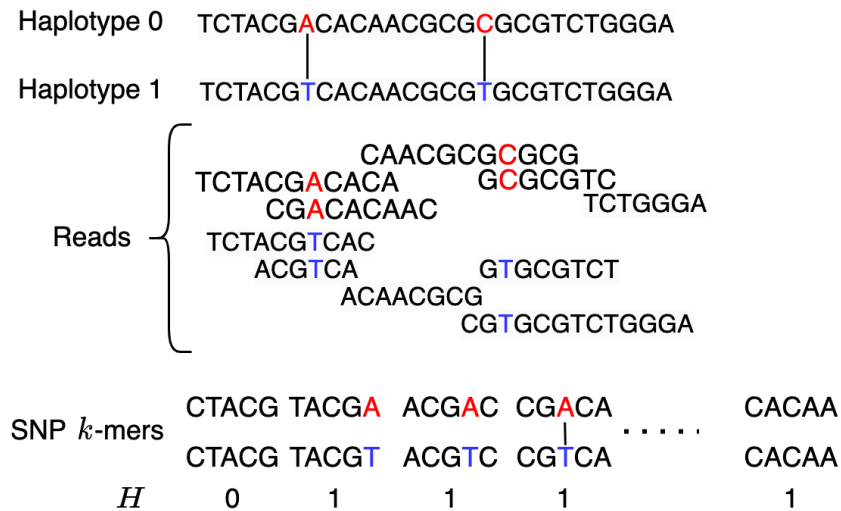


Figure 4.2: An example of Kmer2SNP. red and green labels indicate the heterozygous variants. H denotes the hamming distance of k -mer pairs.

4.3 Heterozygous Contig Phasing

After pre-assembly and Kmer2SNP variant calling, contigs are assembled and SNPs are called. Assembled contigs can be classified to either homozygous, heterozygous (haplotype-specific), or erroneous. In this section, the algorithm identifies the heterozygous contig pairs and phase them into two bins using HiC reads.

4.3.1 Identifying heterozygous contig pairs

Build SNP graph

A SNP graph $G_{snp} = (V, E)$ is an undirected edge weighted graph, where each vertex $v \in V$ represents an assembled contig. Two vertices $u, v \in V$ are connected by an edge if they are supported by the SNPs. The algorithm first constructs the edge set for the SNP graph. Given a SNP k -mer pair as $P_i = \{p_0^i, p_1^i\}$, where p_0^i (p_1^i) consists of adjacent overlapping heterozygous k -mers. Since k -mer size is predefined, we can convert the k -mer into a $2k$ -bit array, where the alphabets (A, C, G, T) is transformed to 2-bit representations (00, 01, 10, 11). Based on the double strandedness feature of genome sequence, Let $R : \{A, C, G, T\} \rightarrow \{A, C, G, T\}$ be the function to reverse complement an alphabet, we have $R(A) = T$, $R(T) = A$, $R(C) = G$, $R(G) = C$. Since a genome sequence can be by either its forward strands or reverse strands ambiguously, we consider the canonical representation of k -mers to eliminate such ambiguity. The canonical representation of a k -mer $s = a_1 a_2 \dots a_k$ is defined as the minimum of s versus $\bar{s} = R(a_k) R(a_{k-1}) \dots R(a_1)$. The algorithm constructs a hash table with key as the canonical bit-array representation of all SNP k -mers, and the value stores the indices for back-tracing the corresponding SNP pair. Such canonical conversion can speed up the query by utilising the hash function without doubling the memory space for hash table. Then, for every contig c , all k -substrings from c is extracted, converted to canonical bit-array representation, and queried against the hash table. The query results describe the mapping between contigs and SNP k -mers. Afterwards, for each SNP k -mer pair $P_i = \{p_0^i, p_1^i\}$, if there exists two distinct contigs that be mapped by k -mers in p_0^i and p_1^i , respectively, we add a *SNP edge* (or increment the edge weight by 1) between them to reflect their likelihood to be heterozygous contig pair at the same position of a chromosome but from different haplotype. The algorithm iterates until all the SNP k -mer pairs are considered. Notice that the algorithm only adds an edge if the mapping is unique to the selected contig pair, which minimizes the chances of adding false-positive edges due to either assembly errors or the false-positive called SNPs.

Find maximum matching on SNP graph

After the SNP graph be constructed, we observe that one heterozygous contig would connect to more than one opposite heterozygous contig, due to SNP calling errors or tandem duplication. In addition, we observe that the edge weight is proportional to the confidence of forming a heterozygous contig pair. Based on that, the algorithm attempts to find the maximum weighted edge matching in the SNP graph. In graph theory, an edge matching is defined as a subset of edges $E' \subseteq E$ that contains a group of pairwise non-adjacent edges, where no two edges in E' can share common vertices. A maximum edge matching is defined as a matching that contains the largest possible number of edges, where a maximum weighted edge matching is defined as a matching that maximize the sum of edge weight among all other matching in the graph. Since maximum weighted edge matching can also be solved in polynomial time, our algorithm solves the maximum weighted edge matching problem via Blossom algorithm [Edmonds](#)

(1965) in the SNP graph, and obtains the most confident SNP edges. Every edge from the matching is also referred to a pair of heterozygous contigs.

4.3.2 Phasing heterozygous contig pairs

Build HiC graph

After obtaining the heterozygous contig pairs, we would like to phase the heterozygous contig pairs into two bins that corresponds to the heterozygous regions of the two haplotypes. Recall that HiC data is error-prone, where the majority of the error is coming from the HiC linkage information between two different haplotypes within the same chromosome. HiC segment spans within the same haplotype among one or more chromosomes are considered to be *informative*, *erroneous* otherwise. An example of erroneous and informative HiC segments is provided in Figure 4.3.

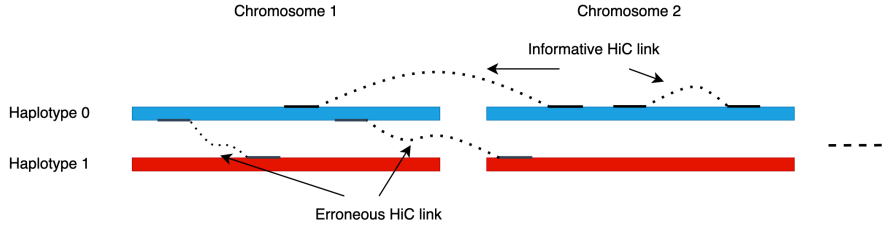


Figure 4.3: An illustration of informative and erroneous HiC links.

We map the HiC segments against all assembled contigs using HiC-Pro [Servant et al. \(2015\)](#), a specialized HiC data processing tool that can process noisy HiC data and generate a HiC contact matrix. Suppose there are n contigs, a HiC contact matrix is a matrix $M = \mathbb{Z}_{\geq 0}^{n \times n}$, where $M[i][j], i \neq j$ represents the number of mapped HiC segments spans between contig c_i and c_j , such HiC segment mapping is also called *HiC link*.

Let $E_{snp} \subseteq V(G_{snp}) \times V(G_{snp})$ denotes the SNP edges from solving the maximum weighted matching problem on SNP graph, a HiC graph $G_{hic} = (V, E)$ is an undirected edge weighted graph, where $V(G_{hic})$ is defined as the union of vertices in E_{snp} . There are two types of edges for G_{hic} , namely HiC (attract) edge and SNP (repel) edge. All the edges from E_{snp} are added to $E(G_{hic})$ and be served as SNP edges (E_{snp}). Given two distinct vertices $u, v \in V(G_{hic})$ representing the contigs c_i and c_j , respectively, where $i \neq j$. A HiC edge ((E_{hic})) $e = (u, v)$ is added to $E(G_{hic})$ if $M[i][j] > 0$. The edge weight $w(e)$ is defined as $M[i][j]$.

If two vertices from the HiC graph are connected by both SNP edge and HiC edge, we only keep the SNP edge and remove the HiC edge. The ignorance of coinciding HiC edges can greatly minimize the majority of noises coming from erroneous HiC links, as described in Figure 4.3.

Find bisection on HiC graph

After the HiC graph is constructed, we aim to find a bisection $V = V_A \cup V_B, V_A \cap V_B = \emptyset$ such that solves the following optimisation problem.

$$\begin{aligned}
T = \min & \sum_{e \in (V_A \times V_B) \cap E(G_{hic})} w(e) \\
\text{s.t.} & (u, v) \notin E_{snp} & \forall u, v \in V_A \\
& (u, v) \notin E_{snp} & \forall u, v \in V_B
\end{aligned}$$

The graph bisection (or partition) problem has been widely studied, and known to be a NP-Hard problem with no known polynomial solution. Therefore, we employ and modify the Kernighan–Lin algorithm [Kernighan and Lin \(1970\)](#) to solve the graph bisection problem in HiC graph that obeys the optimisation constraints.

Before introducing the algorithm and its variation, we describe the concept of internal cost (I -value) and external cost (D -value) with respect to the current bisection, and prove the selection criteria for improving the current bisection result that minimizes optimization cost.

Given the bisection $V = V_A \cup V_B$, the internal cost I_a (I_b) for every vertex $a \in V_A$ ($b \in V_B$) is defined as the following.

$$\begin{aligned}
I_a &= \sum_{e \in \{(a,v) \in \{a\} \times V_A | u \neq v\} \cap E_{hic}} w(e) \\
I_b &= \sum_{e \in \{(b,v) \in \{b\} \times V_B | u \neq v\} \cap E_{hic}} w(e)
\end{aligned}$$

Similarly, the external cost E_a (E_b) for every vertex $a \in V_A$ ($b \in V_B$) is defined as the following.

$$\begin{aligned}
E_a &= \sum_{e \in \{(a,v) \in \{a\} \times V_B | u \neq v\} \cap E_{hic}} w(e) \\
E_b &= \sum_{e \in \{(b,v) \in \{b\} \times V_A | u \neq v\} \cap E_{hic}} w(e)
\end{aligned}$$

Let D_s defines the difference between E_s and I_s for $s \in V$, *i.e.* $D_s = E_s - I_s$.

Operation $flip(a, b)$ is defined as swapping the vertex $a \in V_A$ with $b \in V_B$, where $(a, b) \in E_{snp}$, such that the internal and external costs against relevant vertices will be changed, and also reflects on the optimisation cost. An example of flip operation is given in [Figure 4.4](#).

Lemma 1. *Given the bisection $V = V_A \cup V_B, V_A \cap V_B = \emptyset$ with optimisation cost T_{old} , perform $flip(a, b)$ with $a \in V_A$, $b \in V_B$, and $(a, b) \in E_{snp}$ will result in a new bisection with cost $T_{new} = T_{old} - D_a - D_b$, where $w(a, b) = 0$ if $(a, b) \notin E_{hic}$.*

Proof. Since I_a (E_a) and I_b (E_b) represent the internal (external) cost of vertices $a \in V_A, b \in V_B$ for the current bisection, respectively. By flipping a with b , all the internal edges between a to other vertices from the partition V_A will be switched to external

4.3 Heterozygous Contig Phasing

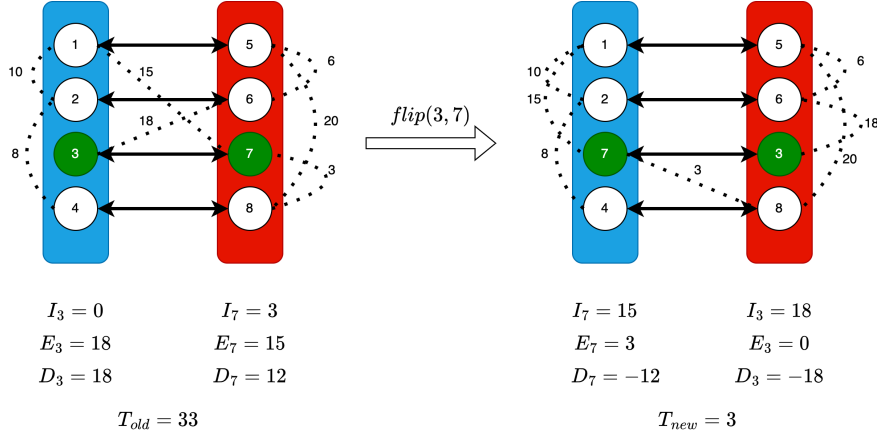


Figure 4.4: An illustration of *flip* operation. Red and blue regions indicate the partition 0 and 1, respectively. dotted lines indicate the HiC links. double-headed lines indicate the SNP edges. Vertex 3 and vertex 7 are flipped and marked green.

edges, and all the external edges that connects a to its alternative partition V_B will be transformed to internal edges. Similarly, the internal and external cost of b will be flipped as well. Therefore, the optimisation cost (total edge weight across the partition) will be reduced by E_a and E_b , but increased by I_a and I_b , as follows.

$$T_{new} = T_{old} - (E_a - I_a) - (E_b - I_b) = T_{old} - D_a - D_b$$

□

Note that the external cost with respect to an edge (a, b) got reduced twice when deducting from E_a and E_b . The reason is the potential edge (a, b) is also served as an external edge that comes across the partition. However, as I mask out any HiC edge that coincides with SNP edge, $w(a, b) = 0$ and can be safely ignored.

Since we have proved that $T_{new} = T_{old} - D_a - D_b$ after performing *flip*(a, b). Equivalently, we have $T_{old} - T_{new} = D_a + D_b$, and the algorithm flip the vertices $a \in V_A$ and $b \in V_B$ if it maximizes $G_{a,b} = D_a + D_b$ at the current iteration, such that minimize the optimisation cost.

Since the optimisation problem restricts that no two vertices connected by a SNP edge can be placed to the same partition, we only consider to flip $a \in V_A$ and $b \in V_B$ if $(a, b) \in E_{snp}$.

The modified Kernighan-Lin algorithm inputs the graph $G_{hic} = (V, E)$ and an initial bisection of $V = V_A^0 \cup V_B^0$ starting with 0-th iteration, and outputs a bisection $V = V_A^i \cup V_B^i$ which minimizes the optimization cost bounded by iteration cutoff m . Algorithm 1

4 Methodology

contains the pseudocode to compute the near-optimal bisection in the HiC graph using modified Kernighan-Lin algorithm.

Notice that the modified Kernighan-Lin algorithm is a local descent algorithm, it attempts to find an improved feasible solution by performing at most $|E_{snp}|$ flip operations per iteration, and doing so with multiple iterations have potential to escape from local minimum, even the G value is negative for some chosen flips at current iteration [Kernighan and Lin \(1970\)](#).

For each iteration, the algorithm takes $O(|V||E|)$ to compute the D -value, $O(|V|/2 \cdot |V|/2) = O(|V|^2)$ (where $|E_{snp}| = |V|/2$ to compute the sequence of flips that maximizes G at each step, and $O(|V|)$ to perform the actual flip to the current bisection. Since the number of iteration is upper bounded by the iteration cutoff m , hence, the time complexity would be $O(m \cdot \max(|V|^2, |V||E|))$. m is 100 by default, however, the algorithm can converge in less than 10 iterations during our experiment.

Since $D_v^i = E_v - I_v$, D^i is inverse proportional to the strength of current partition result $V = V_A^m \cup V_B^m$. For all vertices $v \in V$, if $D_v^i > \delta$ ($\delta = 10$ by default), we manually put v into its alternative partition, which can further minimize the optimisation cost. The reason for still having large D -value after performing the algorithm is due to the false-positive identified SNP pairs, since we only flip the vertices if they are connected by SNP edges. Manually fixing the outliers can mitigate the errors during SNP graph matching and maximize the strength of HiC linkage information.

4.4 Homozygous Contig Phasing

After the bisection on the HiC graph is identified, we have two disjoint sets of heterozygous contigs, C_A and C_B correspond to two different haplotypes, respectively. In this section, we aim to facilitate the homozygous contigs and the remaining unidentified heterozygous contigs into the haplotype-specific partitions using assembly graph and HiC information.

In this thesis, an assembly graph is defined as an undirected node-centric graph $G = (V, E)$, with each vertex $v \in V$ carries a DNA segment $seq(v) \in \{A, C, T, G\}^*$, and each edge $e = \{u, v\} \in E$ represents the variable length end-to-end non-exact overlap between $seq(u)$ and $seq(v)$. Every vertex in assembly graph is also called a contig. Let C denotes all the contigs from the assembly graph, we have $C_N = C \setminus (C_A \cup C_B)$ that did not participate during previous steps, which is more likely to be homozygous contigs, or unidentified heterozygous contigs with small number of variations.

4.4.1 Label propagation

Let $Label : V \rightarrow \{A, B, N\}$ denotes the label of vertices in the assembly graph, since we have C_A and C_B representing the phase of the identified hemizygous (or haplotype-specific) contigs, we can assign the haplotype-specific label A and B to vertices in V if

Algorithm 1 Compute the near-optimal bisection in the HiC graph

Input: G_{hic} , $V = V_A^0 \cup V_B^0$, E_{snp} , m **Output:** $V = V_A^m \cup V_B^m$, D^i

```

1:  $i \leftarrow 0$ 
2: while  $m \neq 0$  do
3:    $D^i \leftarrow \{\}$  ▷ Compute D values
4:   for  $v \in V(G_{hic})$  do
5:      $I_v \leftarrow 0$ 
6:      $E_v \leftarrow 0$ 
7:     for  $e = (u, v) \in adj(v)$  do
8:       if  $u \in V_A^i \wedge v \in V_A^i$  then
9:          $I_v \leftarrow I_v + w(e)$ 
10:      else if  $u \in V_B^i \wedge v \in V_B^i$  then
11:         $I_v \leftarrow I_v + w(e)$ 
12:      else
13:         $E_v \leftarrow E_v + w(e)$ 
14:      end if
15:    end for
16:     $D_v^i \leftarrow E_v - I_v$ 
17:  end for
18:   $G \leftarrow \square$ 
19:   $V \leftarrow \square$ 
20:   $S \leftarrow E_{snp}$ 
21:  for  $j$  such that  $1 \leq j \leq |E_{snp}|$  do
22:     $(a_{max}, b_{max}) \leftarrow \arg \max_{(a,b) \in S} (D_a^i + D_b^i)$  ▷ Pick a flip pair maximize G value
23:     $S \leftarrow S \setminus \{(a_{max}, b_{max})\}$ 
24:     $G.insert(D_{a_{max}}^i + D_{b_{max}}^i)$ 
25:     $V.insert((a_{max}, b_{max}))$ 
26:    for  $v \in V(G_{hic}) \setminus \{a_{max}, b_{max}\}$  do
27:      Update  $D_v^i$ 
28:    end for
29:  end for
30:   $g_{max} \leftarrow \max_{g \leq |G|} \sum_{k=1}^g G[k]$ 
31:  if  $g_{max} > 0$  then
32:    for  $(a, b)$  in  $V$  do
33:      swap  $a \in V_A^i$  with  $b \in V_B^i$  ▷ Perform actual flip operation
34:    end for
35:     $i \leftarrow i + 1$ 
36:  else return  $(V_A^i, V_B^i, D^i)$ 
37:  end if
38: end while

```

4 Methodology

it corresponds to the contigs from C_A and C_B , respectively. The remaining unassigned vertices will have undefined label N by default.

A vertex $v \in V$ is *simple* if $\text{degree}(v) \leq 2$, and *non-simple* otherwise. An edge $e = (u, v) \in E$ is *simple* if either u or v is simple. The label propagation algorithm propagates the label through simple edges. If both endpoint of a simple edge carry defined labels, we do not perform propagation for this edge.

Algorithm 2 contains the pseudocode to compute the label propagation in the assembly graph using iterative approach. For instance, the algorithm first computes all the simple vertices based on degrees. Then, it iteratively propagates the defined label to the adjacent simple undefined vertices, and terminates until no more propagation can be applied to the graph.

Algorithm 2 Compute the label propagation in the assembly graph

Input: G, Label

Output: Label

```

1:  $\text{simple} \leftarrow []$ 
2: for  $v \in V(G)$  do
3:   if  $\text{Degree}(v) \leq 2$  then
4:      $\text{simple}[v] \leftarrow \text{True}$ 
5:   else
6:      $\text{simple}[v] \leftarrow \text{False}$ 
7:   end if
8: end for
9:  $p \leftarrow -1$ 
10: while  $p \neq 0$  do
11:    $p \leftarrow 0$ 
12:   for  $(u, v) \in E(G)$  do
13:     if  $\text{simple}[u] \wedge \text{Label}(u) = N \wedge \text{Label}(v) \neq N$  then
14:        $\text{Label}(u) \leftarrow \text{Label}(v)$ 
15:        $p \leftarrow p + 1$ 
16:     end if
17:     if  $\text{simple}[v] \wedge \text{Label}(v) = N \wedge \text{Label}(u) \neq N$  then
18:        $\text{Label}(v) \leftarrow \text{Label}(u)$ 
19:        $p \leftarrow p + 1$ 
20:     end if
21:   end for
22: end while

```

The label propagation computes all the simple vertices in $O(V)$. Then, since the label can only be propagated to an undefined adjacent simple vertex, there are at most V number of propagation, each takes $O(1)$. Overall, the label propagation algorithms runs in $O(V)$. Figure 4.5 provides an example of label propagation.

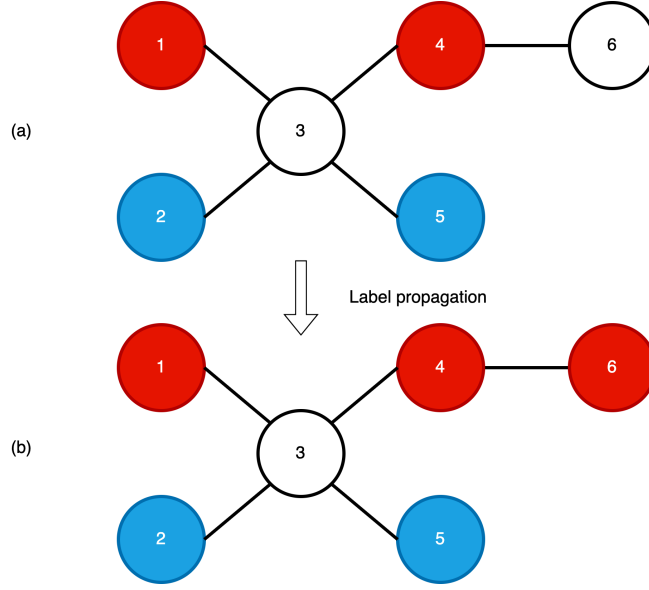


Figure 4.5: An illustration of label propagation. The red and blue colored vertices carry the label A and B , respectively. After label propagation, simple vertex 6 is assigned with label A , vertex 3 is non-simple and keeps the label N .

4.4.2 Steiner Tree Formulation

After the labels be assigned and propagated onto the assembly graph though label propagation, we aim to connect the vertices carrying same label by traversing the graph though vertices with undefined label, which builds a network on the graph. The network is constructed using the idea of Steiner tree.

Steiner tree problem in graphs

A tree $T = (V, E)$ is defined as an connected acyclic undirected graph. Given an undirected graph $G = (V, E)$ with non-negative edge weight, and a subset of vertices $T \subseteq V$ (*terminals*). The Steiner tree problem solves for a tree T in the graph G that at least contains all the terminals T by minimizing the total weight of connected edges.

Notice that the Steiner tree problem is NP-hard, therefore, we employ an approximation algorithm [Takahashi and Matsuyama \(1980\)](#) to solve the Steiner tree problem, with approximation ratio of $2 - 2/|T|$. Given the graph $G = (V, E)$, the edge weight function w and terminals $T = \{t_1, t_2, \dots, t_{|T|}\}$ with $m = |T|$. First, the shortest paths source from terminals in the graph is pre-computed via Dijkstra algorithm [Dijkstra \(2022\)](#). P denotes the $|V| \times |V|$ matrix that stores the shortest path sources from terminals, and D denotes the $|V| \times |V|$ distance matrix that stores the total edge weight along each shortest path.

Then, the algorithm constructs a tree $S = \{t_1\}$ containing a selected terminal t_1 as

4 Methodology

the root. Let $s_t \in S$ denotes the endpoint of the minimum weighted path $P[s_t][t] = (s_t, \dots, t)$, as follows.

$$\forall t \in T. s_t = \arg \min_{s \in S} D[s][t]$$

Also, let t_{min} denotes the unselected terminal that contains the minimum weighted path among all unselected terminals $T \setminus S$, as follows.

$$t_{min} = \arg \min_{t \in T \setminus S} D[s_t][t].$$

Starting with the tree S , in every iteration, the algorithm will compute t_{min} , and add t_{min} to S , together with the visited vertices along the path $P[s_{t_{min}}][t_{min}]$, such that minimizes the cost on appending a new terminal vertex to S at current iteration. The algorithm repeats until all the terminal vertices be added to S .

Solve Steiner tree problem in the assembly graph

Back to the assembly graph $G = (V, E)$, we can partition the vertices $V = V_A \cup V_B \cup V_N$, $V_A \cap V_B \cap V_N = \emptyset$ based on the assigned labels. We aim to separately form Steiner trees to connect vertices in V_A and V_B via vertices in V_N , respectively.

Recall the HiC contact matrix $M = \mathbb{Z}_{\geq 0}^{n \times n}$ stores the HiC links between every pair of contigs in the assembly graph, let c_v denotes the contig carried by vertex $v \in V$, we define $w_A : V \rightarrow \mathbb{Z}_{\geq 0}$ and $w_B : V \rightarrow \mathbb{Z}_{\geq 0}$ as the vertex weight, and be assigned according to M , V_A , and V_B , as follows.

$$\begin{aligned} \forall v \in V. w_A(v) &= \sum_{(u,v) \in V_A \times \{v\}} M[c_u][c_v] \\ \forall v \in V. w_B(v) &= \sum_{(u,v) \in V_B \times \{v\}} M[c_u][c_v] \end{aligned}$$

Similarly, we define the edge weight $W_A : E \rightarrow \mathbb{Z}_{\geq 0}$ and $W_B : E \rightarrow \mathbb{Z}_{\geq 0}$. $\forall e = (u, v) \in E(G)$, we have $W_A(e) = w_A(u) + w_A(v)$ and $W_B(e) = w_B(u) + w_B(v)$.

Since the assembly graph may not be connected in practice, consider a connected induced subgraph $G' = (V', E')$, where $V' \subseteq V$ and $E' \subseteq E$, and we would like to connect all the vertices $V_A \cap V'$ (or $V_B \cap V'$) from the subgraph G' by solving the Steiner tree problem. Given G' and V_p , where $p \in \{A, B\}$, the algorithm temporarily remove all the vertices (and adjacent edges) from $V_{\bar{p}} \cap V'$, which results in a group of smaller connected subgraph F_p induced from G' .

Notice the prior approximation algorithm only works for non-negative edge weights due to the constraint on Dijkstra algorithm. Hence, $W_{\bar{p}}$ is assigned to $f_p \in F_p$, and the goal is to minimize the HiC links within the tree versus the vertices from alternative label $\bar{p}$.

For each smaller subgraph $f_p \in F_p$, we find a Steiner tree T_f with respect to the terminals $V(f_p) \cap V_p$, which connects all terminals via vertices with undefined labels that has minimum HiC link to alternative label $\bar{p}$. Algorithm 3 describes the pseudocode to solve the Steiner tree problem in a subgraph $f_p \in F_p$ with respect to label p .

Algorithm 3 Compute the Steiner tree problem in a subgraph

Input: $f_p, p, V_p, W_{\bar{p}}$ **Output:** T_p

- 1: $S_{f,p} \leftarrow V(f_p) \cap V_p$
 - 2: $(D, P) \leftarrow \text{shortest_paths}(f_p, W_{\bar{p}}, S_{f,p})$ ▷ Find the shortest paths
 - 3: $T_{f,p} \leftarrow \{\arg \max_{t \in S_{f,p}} |\text{seq}(c_t)|\}$
 - 4: $S_{f,p} \leftarrow S_{f,p} \setminus T_{f,p}$
 - 5: **while** $S_{f,p} \neq \emptyset$ **do**
 - 6: $\forall t \in T. s_t = \arg \min_{s \in T_{f,p}} D[s][t]$
 - 7: $t_{\min} = \arg \min_{t \in S_{f,p}} D[s_t][t]$.
 - 8: $\forall v \in P[s_{t_{\min}}][t_{\min}]. T_{f,p} \leftarrow T_{f,p} \cup \{v\}$
 - 9: $S_{f,p} \leftarrow S_{f,p} \setminus \{t_{\min}\}$
 - 10: **end while**
 - 11: **return** $T_{f,p}$
-

The approximate Steiner tree algorithm computes the shortest paths in $O(m|V|^2)$. Then, the algorithm adds at least one terminal into the tree per iteration, where each iteration takes $O(V^2)$ to compute the terminal with minimum shortest path to the tree. Hence, the algorithm runs in $O(m|V|^2)$.

We compute two Steiner trees for each connected subgraph with respect to the two haplotype-specific labels, and add the contigs that be connected within the tree into either bin C_A and C_B with inclusion. For the subgraphs that do not have any assigned label, any contig c in those subgraphs are assigned to the bin C_A or C_B based on the HiC links.

Figure 4.6 describes an example of solving Steiner tree problem in a connected subgraph.

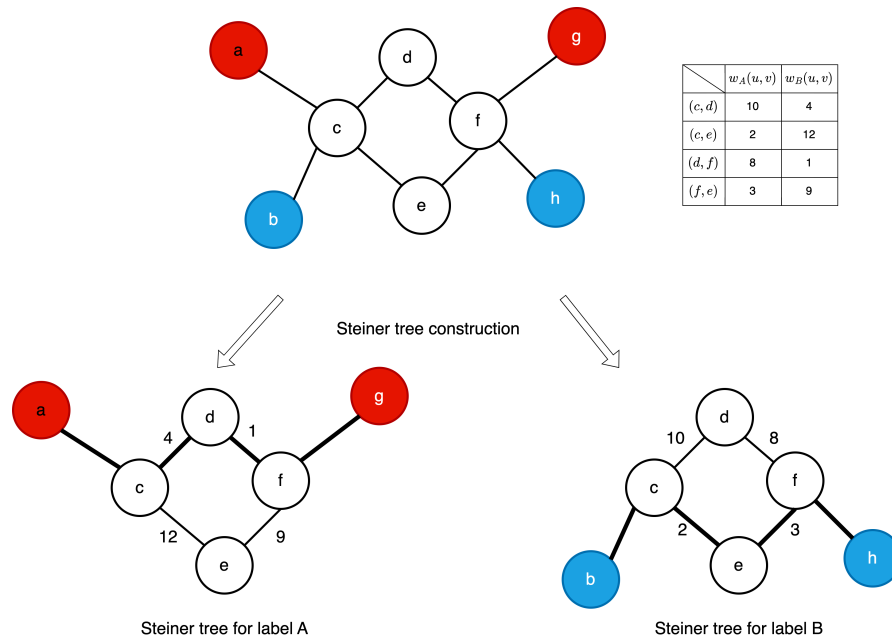


Figure 4.6: An example of solving Steiner tree problem in a connected subgraph. The bold edges are selected by the Steiner tree.

Evaluation

5.1 Experimental Setup

In this section, we describe the datasets¹ be used to evaluate the performance and scalability of our proposed phasing algorithm. Baselines and evaluation metrics are also been discussed.

5.1.1 Experimental dataset

Pt76 dataset

Pt76 refers to an Australian isolate of the leaf rust fungus *Puccinia triticina* (pathotype 76-3,5,7,9,10,12,13), which was collected from wheat cultivar Morocco. The published Pt76 dikaryotic genome [Duan et al. \(2022\)](#) contains 18 chromosomes, with the total genome size approximated to 237MB (118.5MB per haplotype).²

The Pt76 real dataset is prepared and sequenced by Schwessinger’s Lab, which consists of Q20 Oxford Nanopore duplex reads (30x per haplotype), Q10 Oxford Nanopore Simplex reads (1x per haplotype), and illumina Hi-C reads (75x per haplotype, with enzymes Sau3AI and MlucI).

Pst104E dataset

Pst104E refers to a single pustule isolate the wheat strip rust fungus *P. striiformis* f. sp. *tritici* (pathotype 104E137A-), collected in 1982. Pst104E is a bio-trophic pathogen that colonizes living hosts and extracts large amounts of nutrients from plant cells. Pst104E genome contains 18 chromosomes, with the total genome size approximated to 147M (72MB per haplotype).

¹Please contact john.luo@anu.edu.au for the unpublished datasets.

²Note that the published assemblies may still have intractable errors.

The Pst104E real dataset is prepared and sequenced by Schwessinger’s Lab, which consists of Q30 Oxford Nanopore duplex reads (20x per haplotype), Q10 Oxford Nanopore duplex reads (40x per haplotype), and illumina Hi-C reads (20x per haplotype, with enzymes DpnII, HinFI, MseI, and DdeI).

5.1.2 Baselines and Evaluation Metrics

Baselines

The proposed algorithm is benchmarked against the state-of-the-art methods on assembling and phasing the dikaryotic genomes including Verkko (Hi-C), Hifiasm (UL)(Hi-C), and GreenHill.

All above baselines under evaluation use default settings unless specified otherwise. Refer to Appendix C for a detailed description of baselines.

Evaluation Metrics

Similar to previous studies, we employ QUAST [Gurevich et al. \(2013\)](#), a genome assemblies evaluation and comparison software widely applied to evaluating the assemblies against the ground-truth reference. For each dataset, we evaluate the baselines against published reference genome via QUAST. We report the genome fraction, duplication ratio, NGA50, error rate, and number of contigs. Detailed description on each summary statistics are given at below.

- Genome fraction: total number of aligned bases³ in the reference, divided by the genome size.
- Duplication ratio: total number of aligned bases in the assembly, divided by the total number of aligned bases in the reference. 1 is optimal ratio.
- NGA50: contig alignment length such that using longer or equal length contigs produce half of the bases of the reference genome.
- Error rate: the sum of mismatch rate, indel rate, and N’s rate (per 100 kbp), which reflects the number of errors with respect to the reference genome size.

To analyse the phasing accuracy, the common approach is to employ Merqury [Rhie et al. \(2020\)](#), a phasing assessment software for evaluating haplotype-specific accuracy using trio data. However, since the parental data is infeasible to be obtained for eukaryotic microorganisms, we instead utilize MUMMER [Delcher et al. \(2003\)](#) to generate a set of ground-truth phased SNP k -mers for each chromosome by aligning against the reference, and analyse the phasing accuracy by counting the number of falsely phased SNPs against the two bins of haplotype-specific contigs.

³A base in the reference genome is counted as aligned if there is at least one contig with at least one alignment to this base.

Denote $chr(i)$ as the i -th chromosome in the genome, a set of ground-truth SNP k -mers S_i is obtained via MUMMER for $chr(i)$. Given a SNP k -mer pair $(s_0, s_1) \in S_i$, we have all the SNP k -mers s_0 coming from the haplotype 0, and all the SNP k -mers s_1 coming from the haplotype 1. For each contig c that aligned to the chromosome $chr(i)$ against reference, we define the x and y coordinate of c as the number of SNP k -mers on haplotype 1 and haplotype 0, respectively. For each baseline, we accumulate all the coordinates and plot a haplotype phasing scatter plot. The optimal phasing results would have contigs that close to the axis and apart from the diagonal.

5.2 Experimental Results

In this section, we show the performance of the proposed method and other baselines on both Pt76 and Pst104E datasets. Since Verkko and Hifiasm are the only two state-of-the-art assemblers that supports both LA and UL reads to generate high quality assembly graph, thus, we employ both Verkko and Hifiasm to perform pre-assembly, and utilize our method to phase the assembled contigs.

5.2.1 Performance on Pt76 Datasets

Table 5.1 summarizes the unphased assembly performance of Verkko and Hifiasm assemblers. Note that both Verkko and Hifiasm support UL and LA reads. Verkko outperforms the Hifiasm in terms of genome fraction, duplication ratio, NGA50, but gets higher error rates.

Table 5.1: The unphased assembly performance on Pt76 real dataset

Assembler	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N's)	#Contigs
Verkko (UL)	97.159%	1.115	2,848,908	26.64	1108
Hifiasm (UL)	95.709%	1.225	1,537,673	20.99	1262

Table 5.2 summarizes the phased assembly performance of Verkko assembler on haplotype 0 and haplotype 1, respectively. For phasing the Verkko mixed assemblies, the native Hi-C phasing method for Verkko gets the worst genome fraction, NGA50, and error rate among all phasing method. Verkko + GreenHill gets best NGA50 since it performs scaffolding explicitly, which ensures more contiguous sequences. However, GreenHill cannot correct the errors that propagates through pre-scaffolding step, which leads to high error rate. On the other hand, our proposed phasing algorithm achieves best phasing accuracy with minimal error rates (11.55 and 17.06) among all the baselines (833.51 and 1519.07 for Verkko (HiC), 247.57 and 246.79 for Verkko+GreenHill).

Table 5.3 summarizes the phased assembly performance of Hifiasm assembler on haplotype 0 and haplotype 1, respectively. For phasing the Hifiasm mixed assemblies, the built-in Hi-C phasing method for Hifiasm has done a great job on both phasing accuracy

5 Evaluation

Table 5.2: The phased assembly performance on Pt76 real dataset for Verkko, Algorithm refers to the proposed phasing algorithm in the thesis. ‘-’, A and B in column H refers to unphased, haplotype 0 and haplotype 1, respectively.

Pt76	H	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N’s)	#Contigs
Verkko	-	97.159%	1.115	2,848,908	26.64	1108
Verkko (HiC)	A	89.678%	1.002	834,506	833.51	29
	B	90.333%	1.038	1,197,277	1519.07	34
Verkko + GreenHill	A	99.078%	1.013	3,721,474	247.57	93
	B	99.274%	1.023	4,222,129	246.79	93
Verkko + Algorithm	A	99.592%	1.024	2,848,908	11.55	138
	B	99.636%	1.031	2,890,236	17.06	175

and contiguity, since it can also utilize the assembly graph to perform scaffolding after phasing the haplotype-specific vertices. Although GreenHill is a phasing and scaffolding algorithm, it slightly increases the NGA50 from raw assemblies (1,537,673 to 1,778,502 on haplotype 1 file), but also leads to significantly high error rate (20.99 to 427.62, 424.4). On the other hand, our phasing algorithm can accurately phase fragmented contigs into two bins with minimal error rate (13.88, 13.28) among all the baselines, even halve the error rate of the most state-of-the-art Hifiasm (UL)(HiC) method (29.59, 29.2).

Table 5.3: The phased assembly performance on Pt76 real dataset for Hifiasm, Algorithm refers to the proposed phasing algorithm in the thesis. ‘-’, A and B in column H refers to unphased, haplotype 0 and haplotype 1, respectively.

Pt76	H	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N’s)	#Contigs
Hifiasm	-	95.709%	1.225	1,537,673	20.99	1262
Hifiasm (HiC)	A	99.742%	1.052	4,493,542	29.59	101
	B	99.788%	1.04	4,339,634	29.2	70
Hifiasm + GreenHill	A	97.1%	1.098	1,311,395	427.62	276
	B	97.849%	1.106	1,778,502	424.4	275
Hifiasm + Algorithm	A	99.415%	1.139	1,318,076	13.88	470
	B	99.678%	1.114	1,724,546	13.28	414

Figure 5.1 refers to the haplotype phasing plot for phasing Verkko assemblies. From the phasing plot, we can see that Verkko (HiC) cannot correctly phase the contigs and tends to mix the haplotype-specific contigs into both bins. In contrast, Verkko+GreenHill do produce more contiguous contigs thanks to its pre-scaffolding step, but also have phasing errors on long contigs, which confirms the higher error rate as shown at Table 5.2. On the other hand, our phasing algorithm is able to adapt the fragmented unphased assemblies generated by Verkko, and phase the contigs correctly into two bins that corresponds to

two haplotypes with minimal switch errors among the baselines.

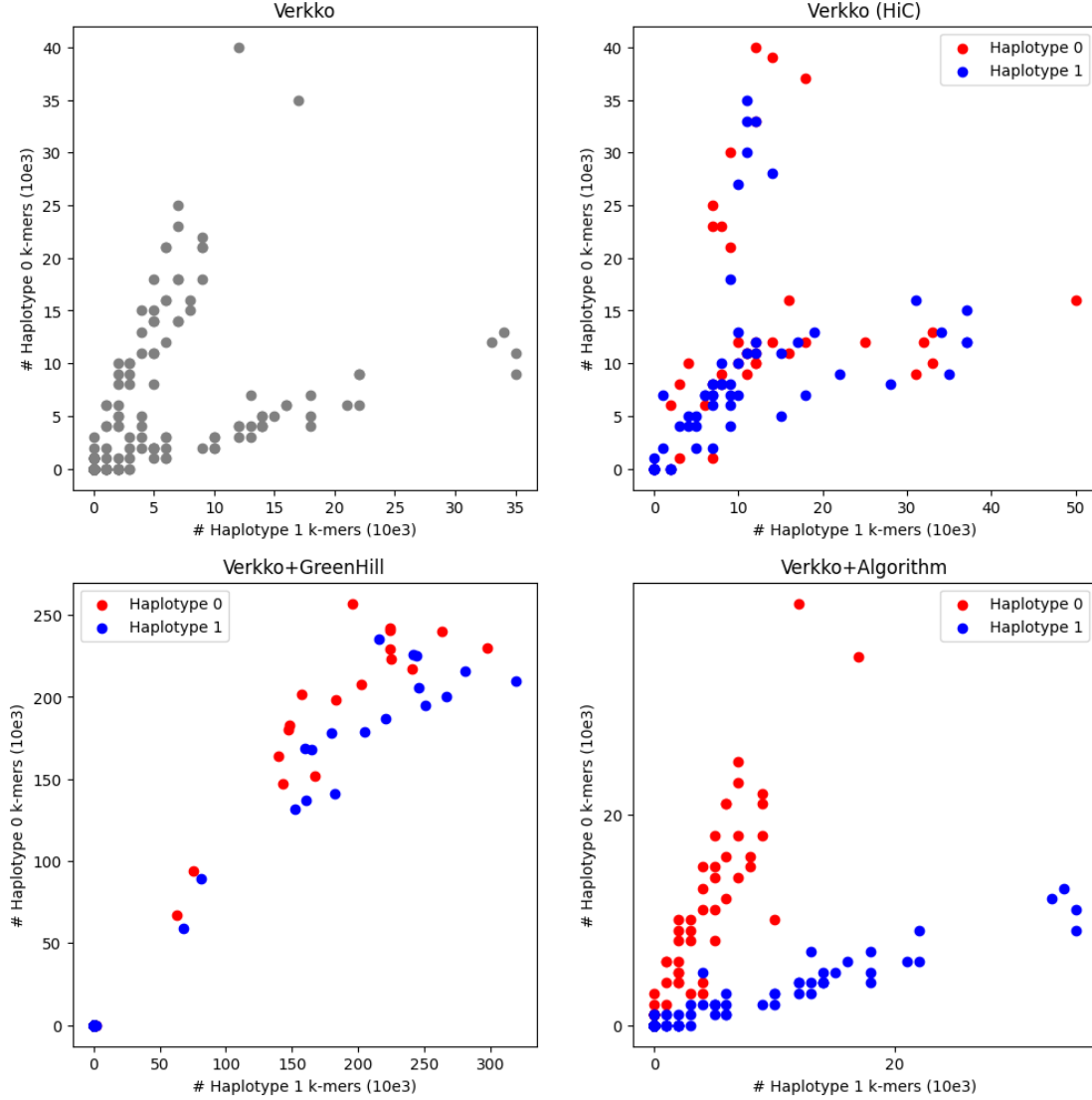


Figure 5.1: The haplotype phasing scatter plot for Verkko assembler on Pt76 dataset. The first plot describes the Verkko unphased assemblies. The other three plots describe the phased accuracy among all baselines. Each point corresponds to a contig. Its coordinate gives the number of Haplotype 0 and Haplotype 1 SNP k -mers on the contig, with these k -mers derived from MUMMER comparing the two published haplotypes of Pt76. Haplotype 0 and 1 refers to the two sets of phased contigs, respectively.

Figure 5.2 refers to the haplotype phasing plot for phasing Hifiasm assemblies. From the phasing plot, we can see that GreenHill tends to produce false-positive concatenated

5 Evaluation

scaffolds by merging SNPs from both haplotypes. Hifiasm can produce more contiguous haplotype-specific contigs, but may occasionally collapse haplotype-specific contigs around the region $[(5,10),(5,10)]$. On the other hand, our method can correctly phase more haplotype-specific contigs among all baselines. It is worth noting that our phasing method can produce consistent result against Hifiasm (HiC) at the region with low heterozygosity, *i.e.* $[(0,2.5),(0,2.5)]$.

To analyse the effectiveness of the Steiner tree method be introduced as the final (optional) step of our phasing algorithm, we visualize the assembly graph by Bandage [Wick et al. \(2015\)](#), an assembly graph visualisation software. Fig 5.3 refers to the labelled graph by choosing the primary contig alignment against ground-truth reference, where Fig 5.4 refers to the labelled graph before and after performing Steiner tree method. From Fig 5.4(a) and Fig 5.4(b), we can see that the Steiner tree method can help to connect the heterozygous contigs coming from the subgraph and leads to higher genome fraction on both bins.

To validate the chromosomal level phasing accuracy, we adapt the cleaned alignment results as part of QUAST, color-code the haplotype-specific contigs, and generate the chromosome-level phasing results using GenomeViz [Ghai et al. \(2004\)](#).

Figure 5.5 is a chromosome-level phasing plot for all baselines against chromosome 3 of Pt76 dataset, where blue, red and gray bars correspond to haplotype 0, haplotype 1, and gaps, respectively. From the plot, we found that GreenHill tends to have long duplicated contigs on both bins, which results in the long red bars on both haplotypes, indicating the potential drawback of pre-scaffolding step. Verkko (HiC) has coverage loss on haplotype 1, which is reflected on lower genome fraction (90.333%) compare to the mixed assemblies (97.159%). On the contrast, both Hifiasm (HiC) and our phasing algorithm can provide accurate phasing results with minimal switch errors. At the left tail region, our phasing algorithm can even reach higher phasing accuracy compare to Hifiasm (HiC). The full chromosome-level phasing results against Pt76 dataset are shown at Appendix A.

It is worth noting that our phasing algorithm is flexible to adapt various assemblers and provide comparable phasing results against all the state-of-the-art baselines.

5.2.2 Performance on Pst104E Datasets

Table 5.4 summarizes the performance of unphased assemblies on Pst104E dataset. Both assemblers produce near-optimal genome fractions, indicating that they can utilize UL reads to separate most heterozygous variants and break most of the bubble vertices. In addition, Verkko assembler consistently produce more contiguous contigs compare to Hifiasm, but also introduces nearly double the error rate.

Table 5.5 and Table 5.6 summarize the phased assembly performance of Verkko and Hifiasm assemblers on haplotype 0 and 1 for Pst104E dataset, respectively. Notice that Verkko (HiC) has really low genome fraction against both haplotypes (49.424%,

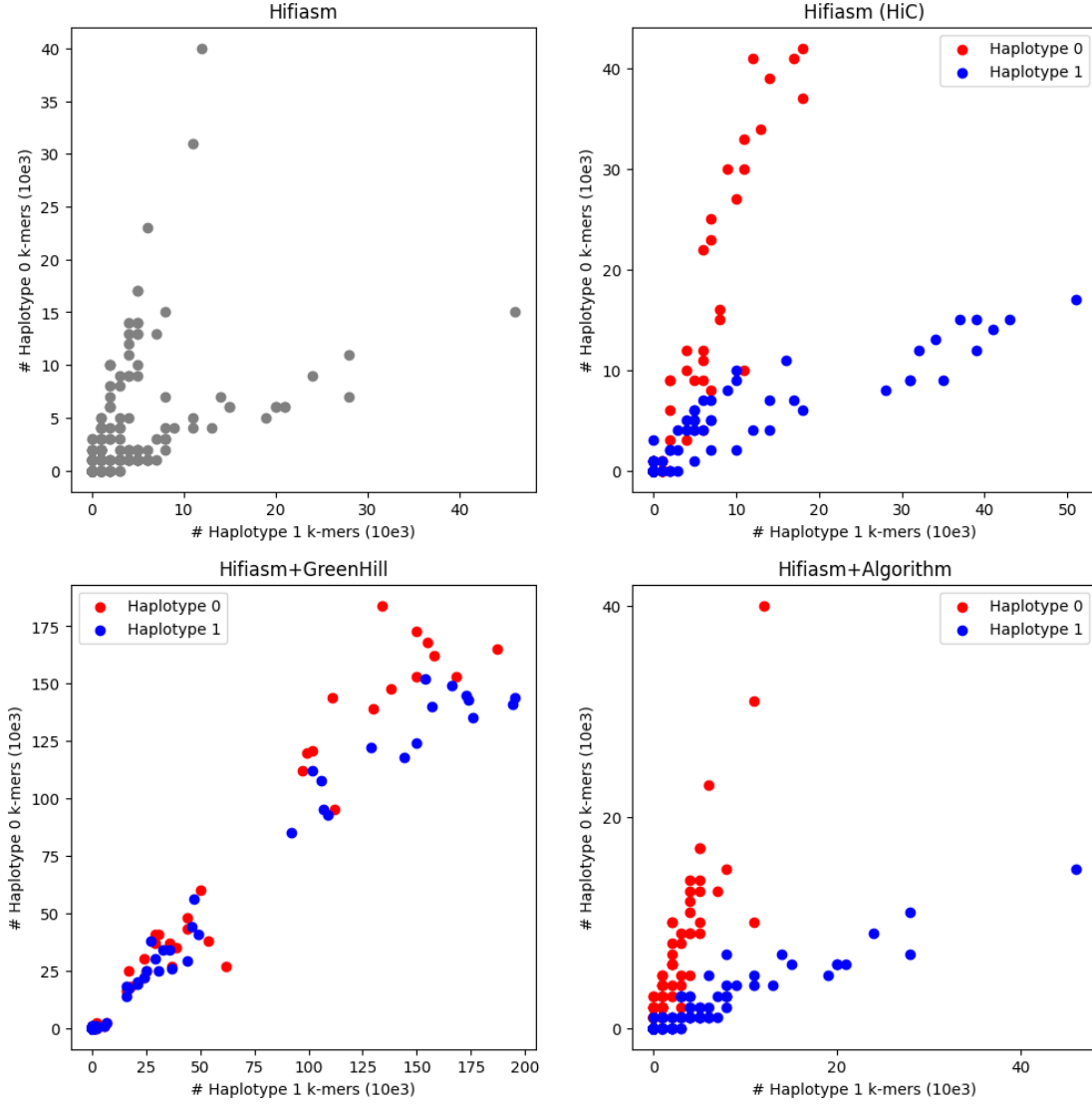


Figure 5.2: The haplotype phasing plot for Hifiasm assembler on Pt76 dataset. The first plot describes the Hifiasm unphased assemblies. The other three plots describe the phased accuracy among all baselines. Each point corresponds to a contig. Its coordinate gives the number of Haplotype 0 and Haplotype 1 SNP k -mers on the contig, with these k -mers derived from MUMMER comparing the two published haplotypes of Pt76. Haplotype 0 and 1 refers to the two sets of phased contigs, respectively.

43.874%) after phasing. By further investigation, we found that Verkko (HiC) has 319 unphased contigs, which indicates that the phasing algorithm for Verkko (HiC) did not fully utilize the Hi-C reads and assembly graph. GreenHill can utilize the Verkko and

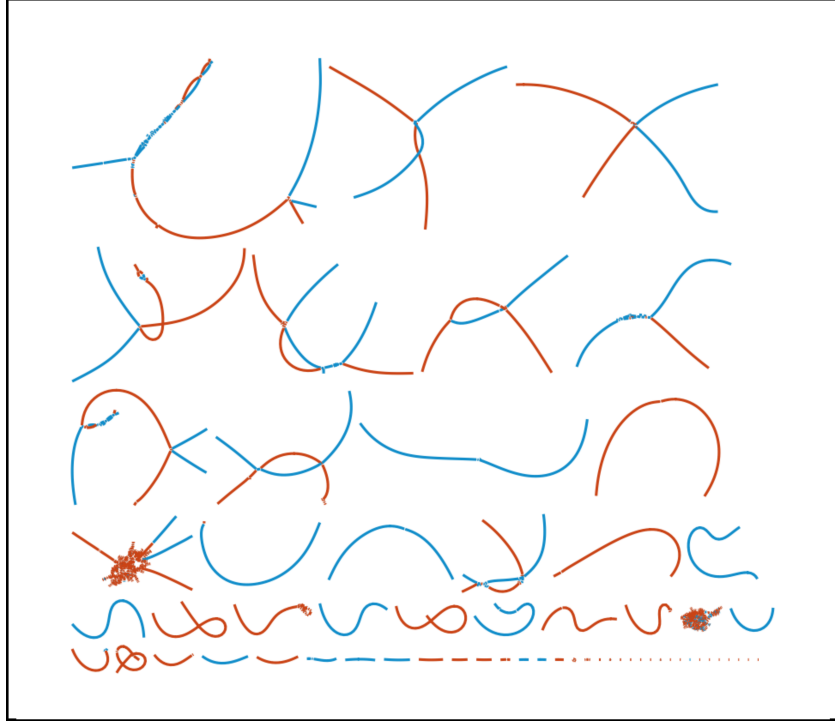


Figure 5.3: Bandage visualisation on the reference labelled assembly graph produced by Verkko assembler on Pt76 dataset. The blue and red labelled contigs refer to contigs from haplotype 0/1, respectively.

Table 5.4: The unphased assembly performance on Pst104E real dataset

Assembler	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N's)	#Contigs
Verkko (UL)	99.946%	1.09	4,542,818	10.42	357
Hifiasm (UL)	99.969%	1.118	2,264,553	4.89	344

Hifiasm assembles and produce the phased contigs with near-optimal genome fraction and NGA50, however, it also leads to high error rate (411.71 and 427.87 for Verkko, 227.27 and 277.11 for Hifiasm). Hifiasm (HiC) can produce the phased results with the best NGA50, since it also implicitly scaffolds the contigs after phasing. Our phasing algorithm can utilize both Hifiasm and Verkko, and produce the phasing results with near-optimal genome fraction and minimal error rates. Notice that our phasing algorithm can even lower the error rate compare to the Verkko mixed assemblies (3.62+3.82 versus 10.42), since our phasing algorithm only consider the selected heterozygous contigs through maximum matching, and homozygous contigs are considered if they have been selected by the Steiner tree. Thus, our algorithm can implicitly ignore the potential erroneous contigs throughout the phasing step and lower the error rate.

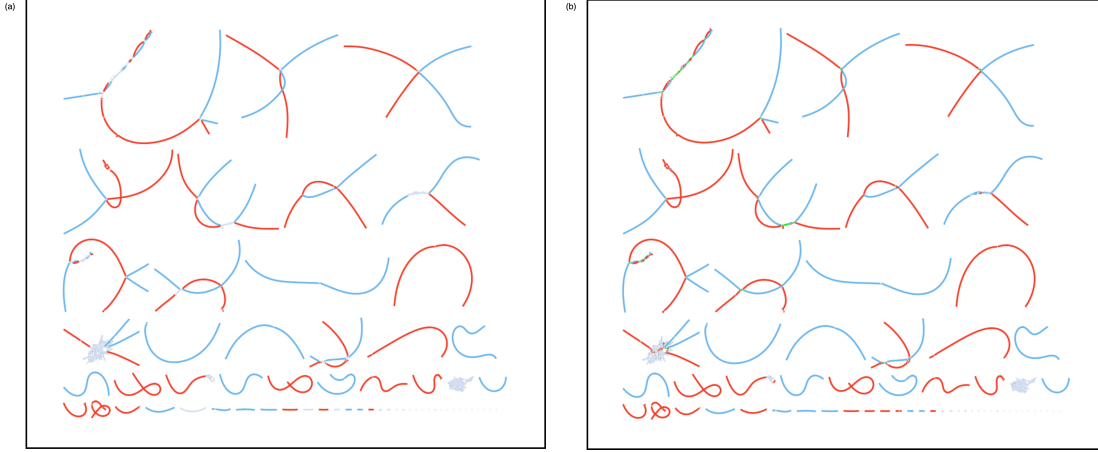


Figure 5.4: Bandage visualisation on the phasing method labelled assembly graph produced by Verkko assembler on Pt76 dataset. (a) The graph before performing the Steiner tree method. (b) The graph after performing the Steiner tree method. The blue and red labelled contigs refer to contigs from phased haplotype 0/1, respectively. The green labelled contigs refer to duplicated contigs into both haplotypes.

Table 5.5: The phased assembly performance on Pst104E real dataset for Verkko, Algorithm refers to the proposed phasing algorithm in the thesis. ‘-’, A and B in column H refers to unphased, haplotype 0 and haplotype 1, respectively. ‘-’ is reported by QUAST on NGA50 if the alignment is too fragmented.

Pst104E	H	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N’s)	#Contigs
Verkko	-	99.946%	1.09	4,542,818	10.42	357
Verkko (HiC)	A	49.424%	1.022	12,167	457.26	20
	B	43.874%	1.003	-	507.89	8
Verkko + GreenHill	A	99.759%	1.023	3,513,079	411.71	32
	B	99.975%	1.04	4,100,000	427.87	32
Verkko + Algorithm	A	99.994%	1.016	4,452,009	3.62	43
	B	99.897%	1.012	4,606,931	3.82	46

Figure 5.6 and Figure 5.7 refer to the haplotype phasing plot for phasing Verkko and Hifiasm assemblies. From the phasing plot, we can see that it reports consistent phasing result compare to Pt76 dataset among all baselines. Verkko (HiC) filters a lot of true-positive heterozygous contigs and leave them unassigned, one potential reason is Verkko is designed for genome with low heterozygosity, and it only allows for genome with divergence rate up to 0.2%, but our Pst104E genome has sequence divergence up to 1.16%, which may leads to difficulty on phasing the high heterozygous regions and result in coverage loss. GreenHill always scaffold contigs belong to alternative haplotypes,

5 Evaluation

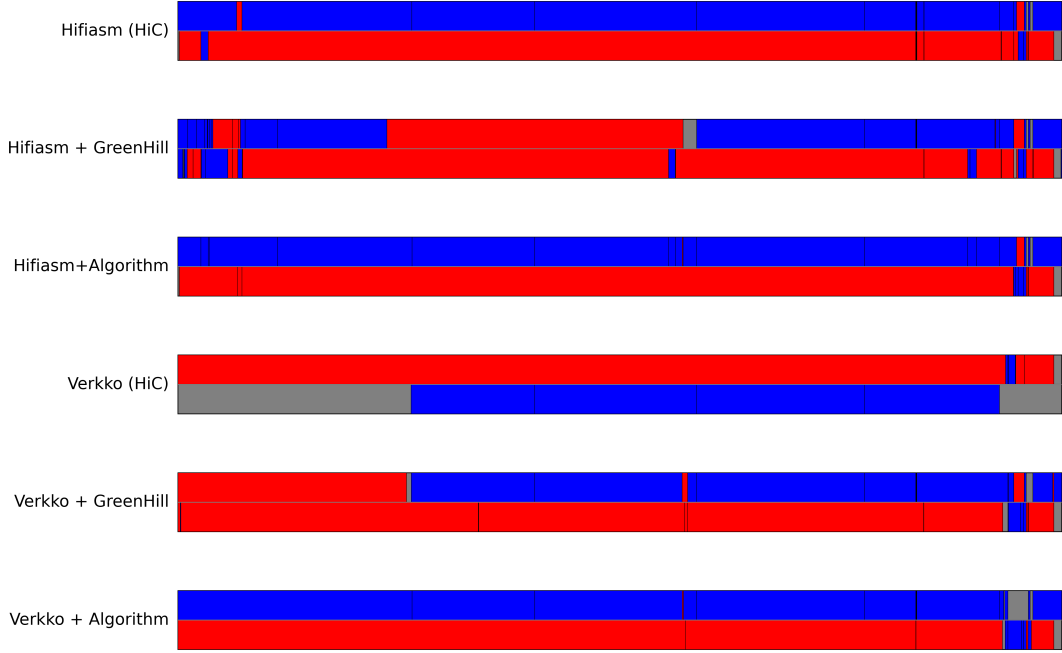


Figure 5.5: Chromosome-level phasing results for all baselines on Chromosome 3 of Pt76 dataset

Table 5.6: The phased assembly performance on Pst104E real dataset for Hifiasm, Algorithm refers to the proposed phasing algorithm in the thesis. -, A and B in column H refers to unphased, haplotype 0 and haplotype 1, respectively.

Pst104E	H	Genome Fraction	Duplication Ratio	NGA50	Error rate (mis+indel+N's)	#Contigs
Hifiasm	-	99.969%	1.118	2,264,553	4.89	344
Hifiasm (HiC)	A	99.814%	1.055	4,452,102	37.84	54
	B	99.833%	1.063	4,606,919	42.45	59
Hifiasm + GreenHill	A	96.963%	1.1	2,363,565	227.27	145
	B	95.91%	1.11	1,930,720	277.11	145
Hifiasm + Algorithm	A	99.973%	1.057	2,631,249	4.05	104
	B	99.566%	1.08	1,986,180	3.57	130

and collapse SNPs from both haplotypes to the same scaffold. On the other hand, Hifiasm (HiC) can accurately phase the contigs with high heterozygosity rate, but may still mis-phase the low heterozygous contigs around $[(0,10),(0,10)]$. Our phasing algorithm produces most accurate phasing result with minimal error rate, which also be reflected on the Figure 5.6(d) and Figure 5.7(d).

The full chromosome-level phasing results against Pst104E dataset are shown at Appendix B.

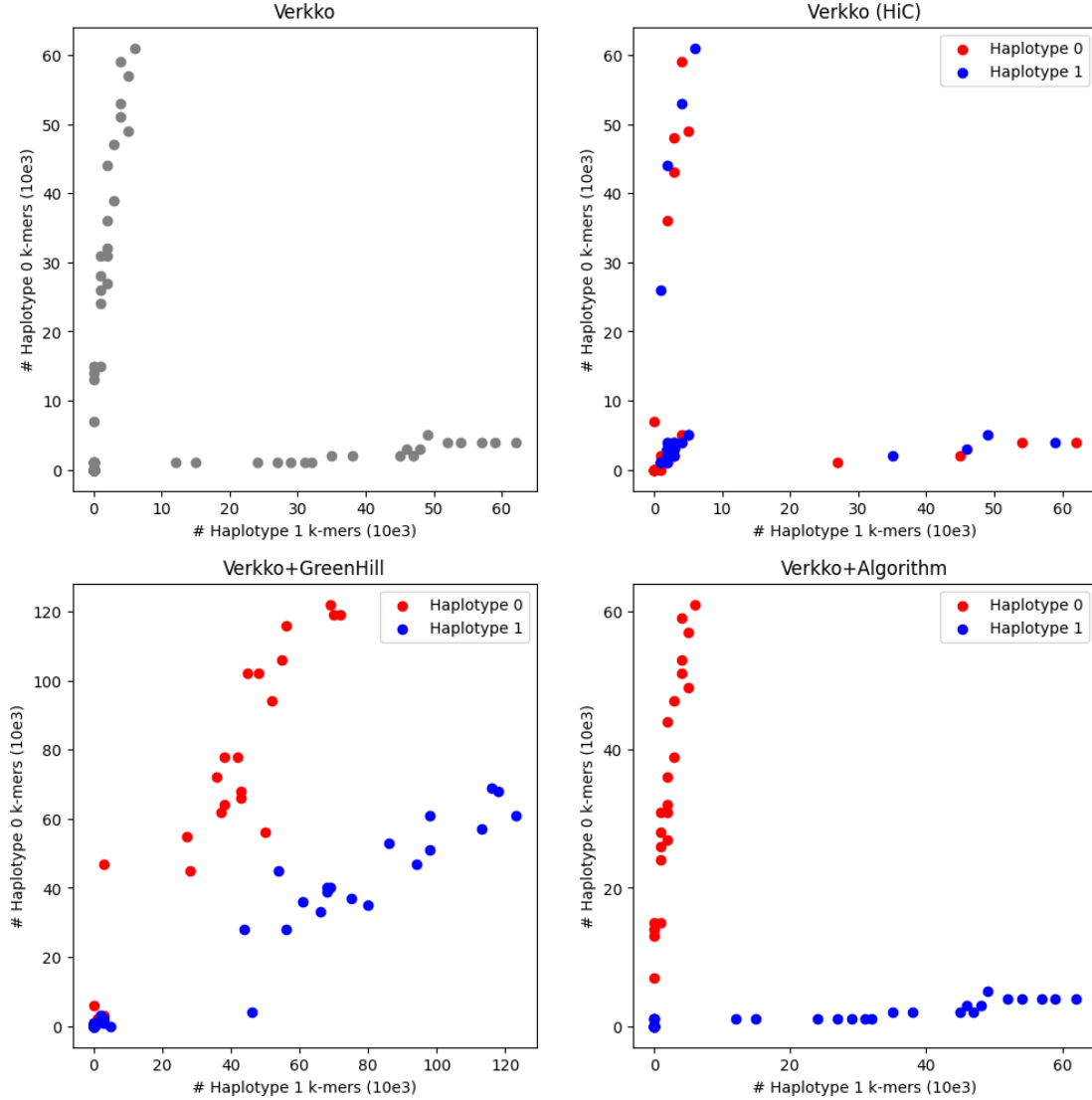


Figure 5.6: The haplotype phasing plot for Verkko assembler on Pst104E dataset. The first plot describes the Verkko unphased assemblies. The other three plots describe the phased accuracy among all baselines. Each point corresponds to a contig. Its coordinate gives the number of Haplotype 0 and Haplotype 1 SNP k -mers on the contig, with these k -mers derived from MUMMER. Haplotype 0 and 1 refers to the two sets of phased contigs, respectively.

5 Evaluation

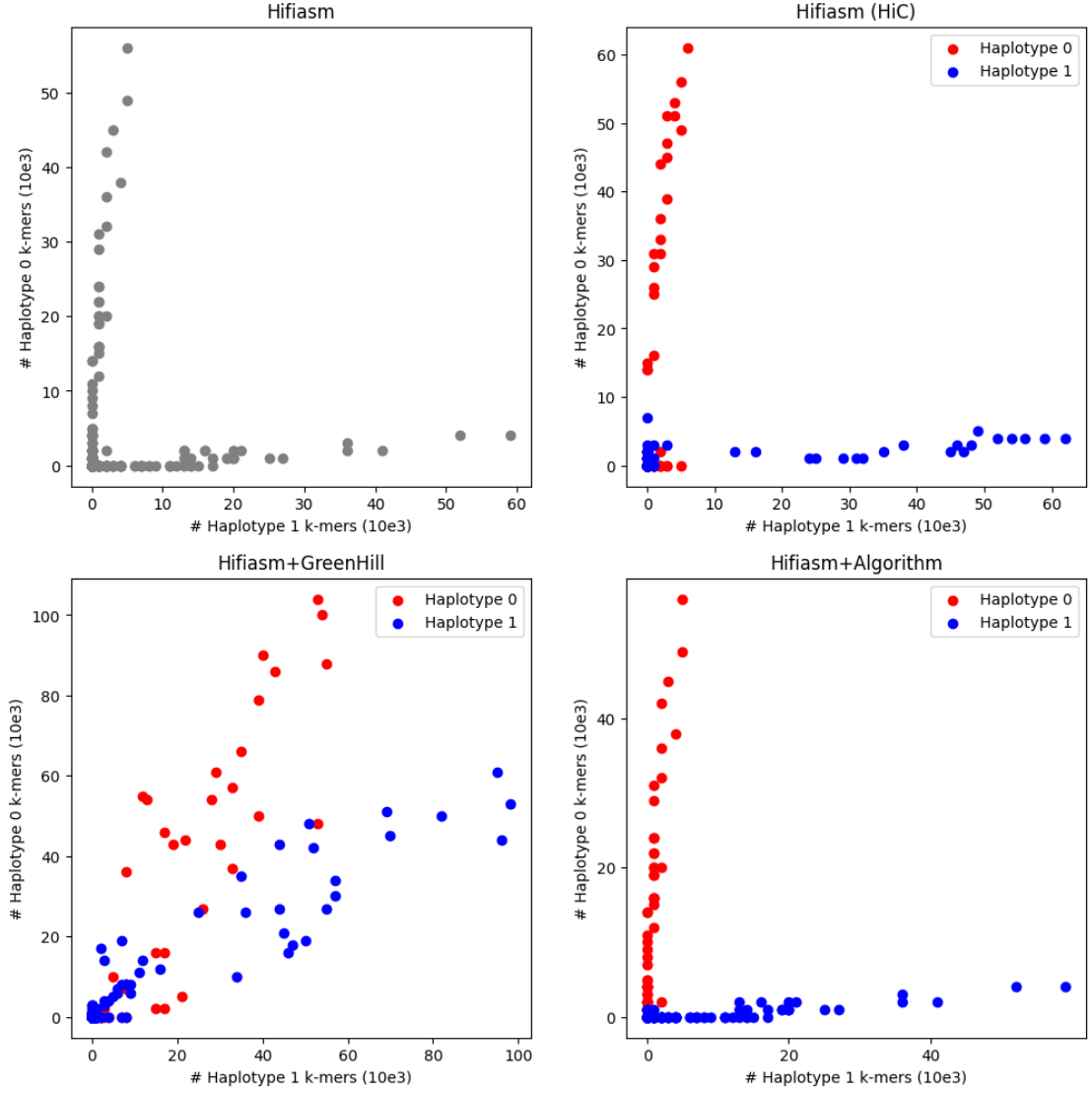


Figure 5.7: The haplotype phasing plot for Hifiasm assembler on Pst104E dataset. The first plot describes the Hifiasm unphased assemblies. The other three plots describe the phased accuracy among all baselines. Each point corresponds to a contig. Its coordinate gives the number of Haplotype 0 and Haplotype 1 SNP k -mers on the contig, with these k -mers derived from MUMMER. Haplotype 0 and 1 refers to the two sets of phased contigs, respectively.

5.2.3 Runtime, Memory and Implementation Details

Table 5.7 summarize the User time and peak memory for different approaches on all benchmarks, respectively. All the experiments were run under the National Computa-

tional Infrastructure (NCI) Gadi supercomputer by submitting jobs to the Gadi biodev queue with the default job dependencies. The allocated RAM size was limited to 500GB and CPU time was limited to 800 hours. The peak memory (maximum resident set size) refers to the peak amount of memory throughout the program execution.

From the Table 5.7, we observe that GreenHill is more efficient than our approach in terms of user time, but may not be a preferred tool due to its extremely high error rate (as shown in Table 5.2, Table 5.3, Table 5.5, and Table 5.6). By incorporation of Hi-C reads, Hifiasm (HiC) can provide phased assemblies without significant overhead with respect to user time and peak memory, thanks to its perfect matching strategy when align the erroneous Hi-C data onto the assembly graph, which utilizes the hash table. On the other hand, our phasing algorithm achieves minimal peak memory usage among all baselines. In addition, the user time of our phasing algorithm is comparable to GreenHill, but achieves much lower switch errors as well.

Table 5.7: User Time and peak memory usage of all baselines. The performance of explicit phasing method (GreenHill and algorithm) is counted without the time and memory usage of pre-assembly step.

	User Time (hours)		Peak memory (GB)	
	Pt76	Pst104E	Pt76	Pst104E
Verkko	21.29	101.69	20.93	25.81
Verkko (HiC)	48.48	116.55	20.94	25.81
Verkko + GreenHill	13.56	13.50	19.55	13.37
Verkko + Algorithm	21.87	14.66	7.00	1.91
Hifiasm	21.74	107.80	25.25	27.18
Hifiasm (HiC)	22.16	107.10	25.94	27.31
Hifiasm + GreenHill	19.35	11.09	19.70	12.98
Hifiasm + Algorithm	19.72	13.89	7.0	1.91

A detailed break down of user time for our phasing algorithm implementation is given at Table 5.8. We observe that most of the computational overhead belongs to the Hi-C alignment step, where our phasing algorithm utilize HiC-Pro to perform Hi-C to contig alignments. In HiC-Pro, it employs Bowtie2 to align the Hi-C reads in a paired-end

Table 5.8: User Time break down for the phasing algorithm pipeline.

	User Time (hours)			
	Pt76		Pst104E	
	Verkko	Hifiasm	Verkko	Hifiasm
HiC-Pro	21.32	19.16	14.52	13.75
Kmer2SNP	0.52	0.52	0.13	0.13
Phasing	0.03	0.04	0.01	0.01

5 *Evaluation*

short reads manner. Since Hi-C reads is erroneous and high density (36GB for Pt76 and 6.1GB for Pst104E), it would be time inefficient to use accurate short read alignment method to perform Hi-C read alignment.

Concluding Remarks

6.1 Conclusion

The advance of sequencing technology benefits the development of telomere-to-telomere assemblers. Phasing haplotypes and discovering genomic variants are necessary for biomedical research [Crawford and Nickerson \(2005\)](#). In this thesis, we introduce a novel haplotype phasing algorithm to phase assembled contigs using HiC reads. Our algorithm is flexible to couple with various state-of-the-art diploid assemblers. Experimental results on both Pt76 and Pst104E rust fungus dataset show that our phasing algorithm achieves the minimal error rates among the state-of-the-art phasing algorithms while maintaining near-optimal genome coverage.

6.2 Future Work

Currently, our phasing algorithm relies on HiC-Pro to perform HiC read alignment, which dominates the running time of our implementation. It is worth exploring how to make the HiC read alignment more efficient. In addition, since our phasing algorithm output both the phased assemblies and the group of Steiner trees that carries the connectivity and orientation information among the contigs, those information can be adapted to perform scaffolding afterwards and generate the full-length haplotypes from the phased assemblies.

Although diploid genome assembly have been studied for many years, no satisfactory solution for polyploid genome be discovered yet [Li and Durbin \(2023\)](#). Our phasing algorithm has the potential to be extended to handle polyploid genome and produce an accurate phased assemblies.

Appendix: Chromosome-level Phasing results for Pt76 Dataset

Figure [A.1- A.3](#) contain the chromosome-level phasing results for Pt76 dataset on phasing Verkko assemblies among all chromosomes. Figure [A.4- A.6](#) contain the chromosome-level phasing results for Pt76 dataset on phasing Hifiasm assemblies among all chromosomes.

A Appendix: Chromosome-level Phasing results for Pt76 Dataset

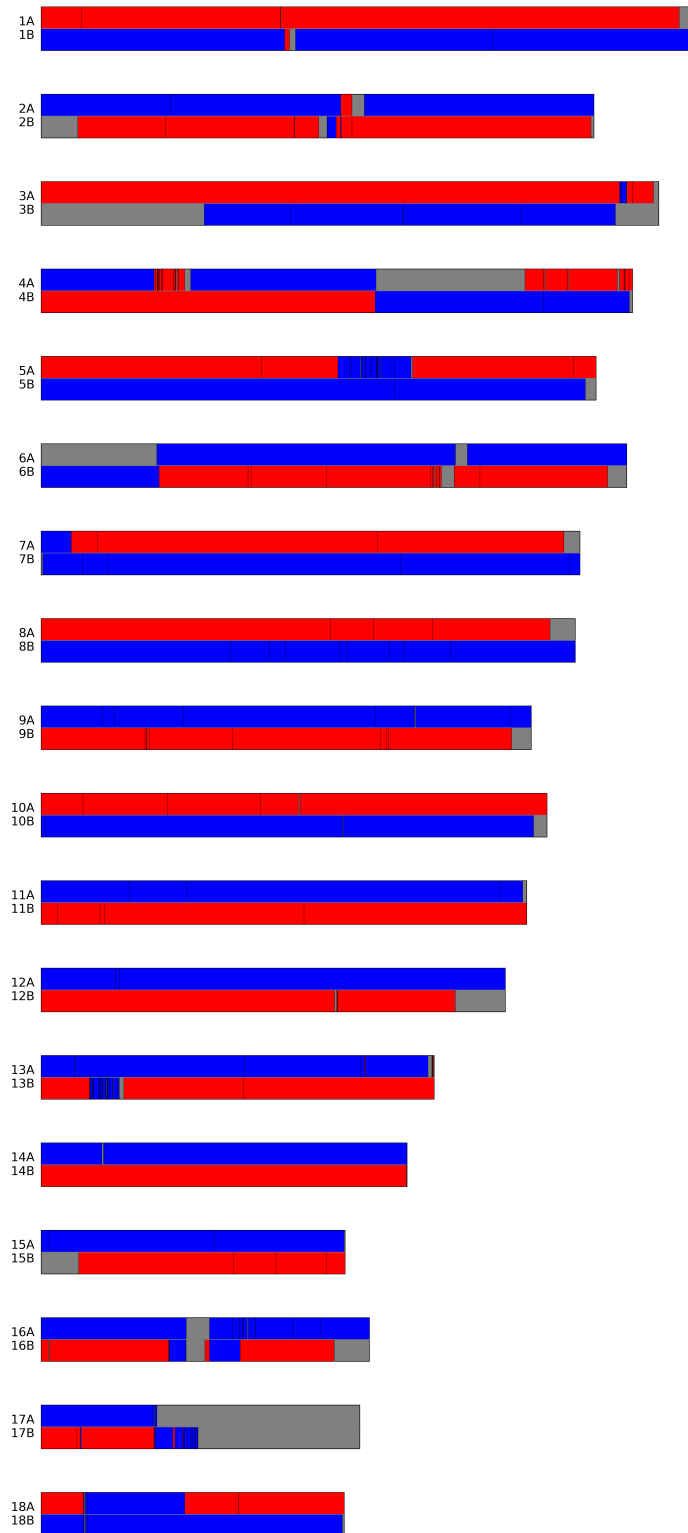


Figure A.1: Chromosome-level phasing results for Verkko (HiC) against Pt76 dataset

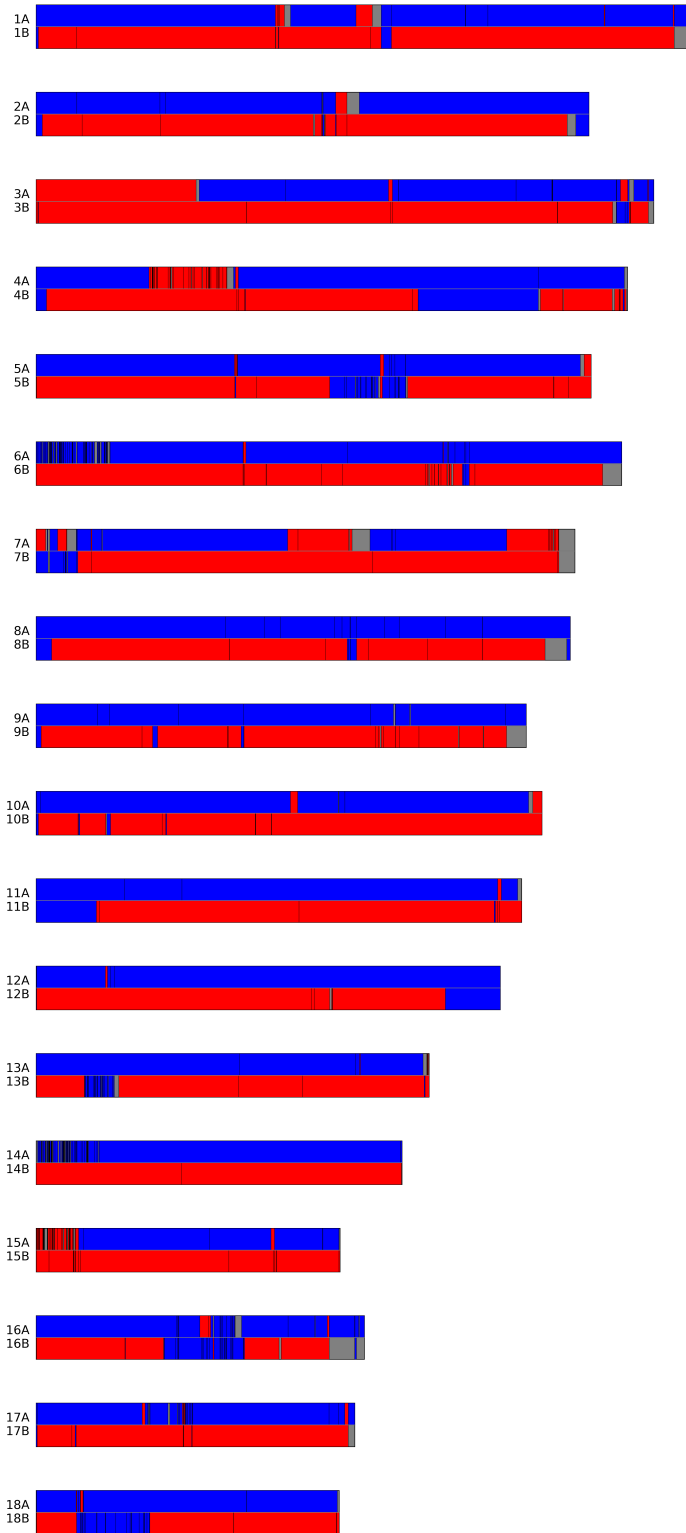


Figure A.2: Chromosome-level phasing results for Verko+GreenHill against Pt76 dataset

A Appendix: Chromosome-level Phasing results for Pt76 Dataset

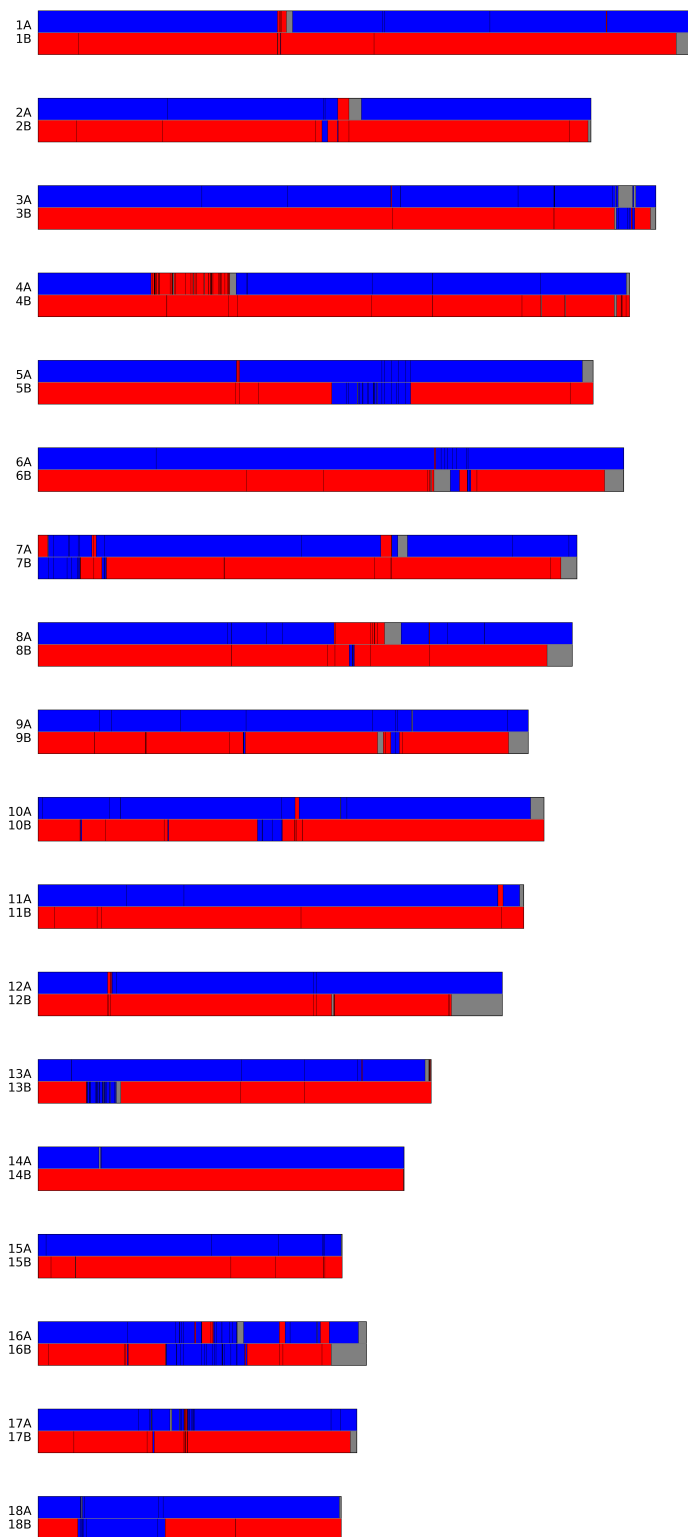


Figure A.3: Chromosome-level phasing results for Verkko+Algorithm against Pt76 dataset

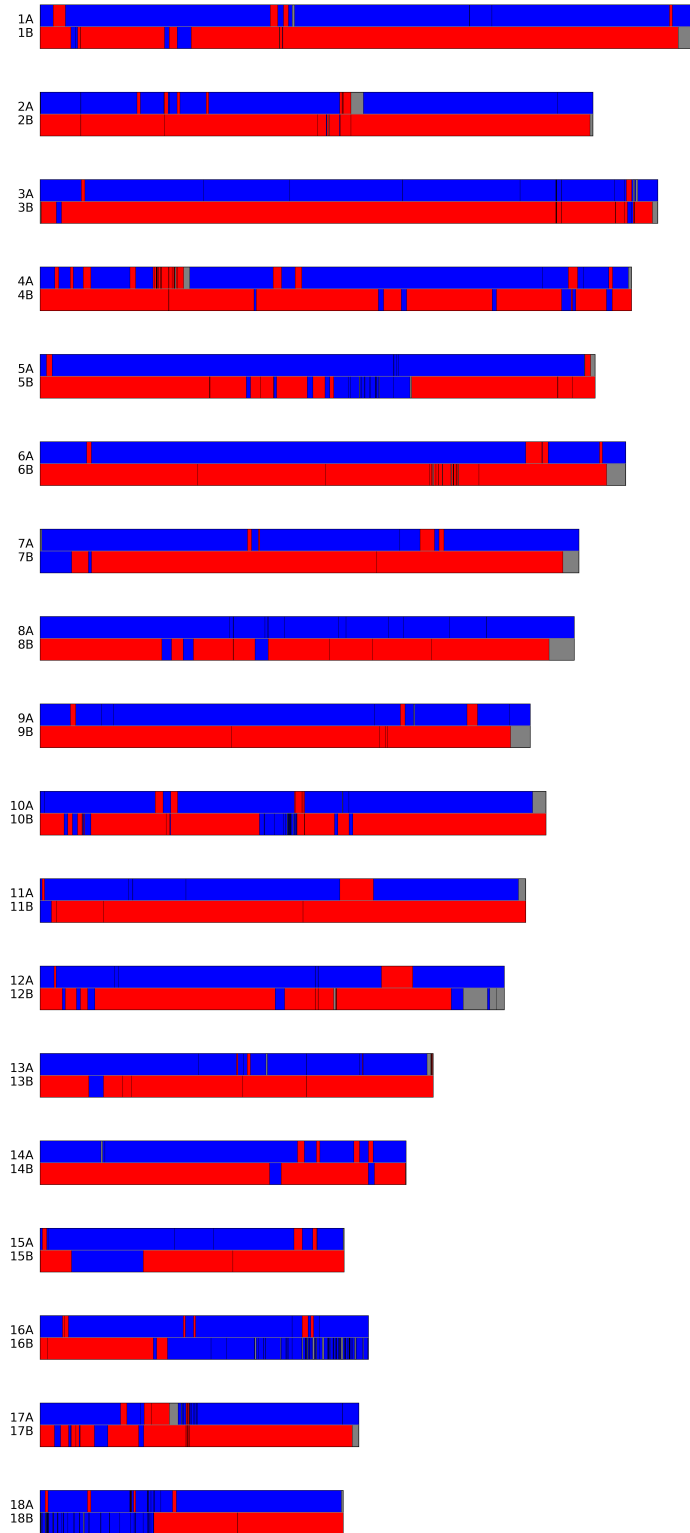


Figure A.4: Chromosome-level phasing results for Hifiasm (HiC) against Pt76 dataset

A Appendix: Chromosome-level Phasing results for Pt76 Dataset

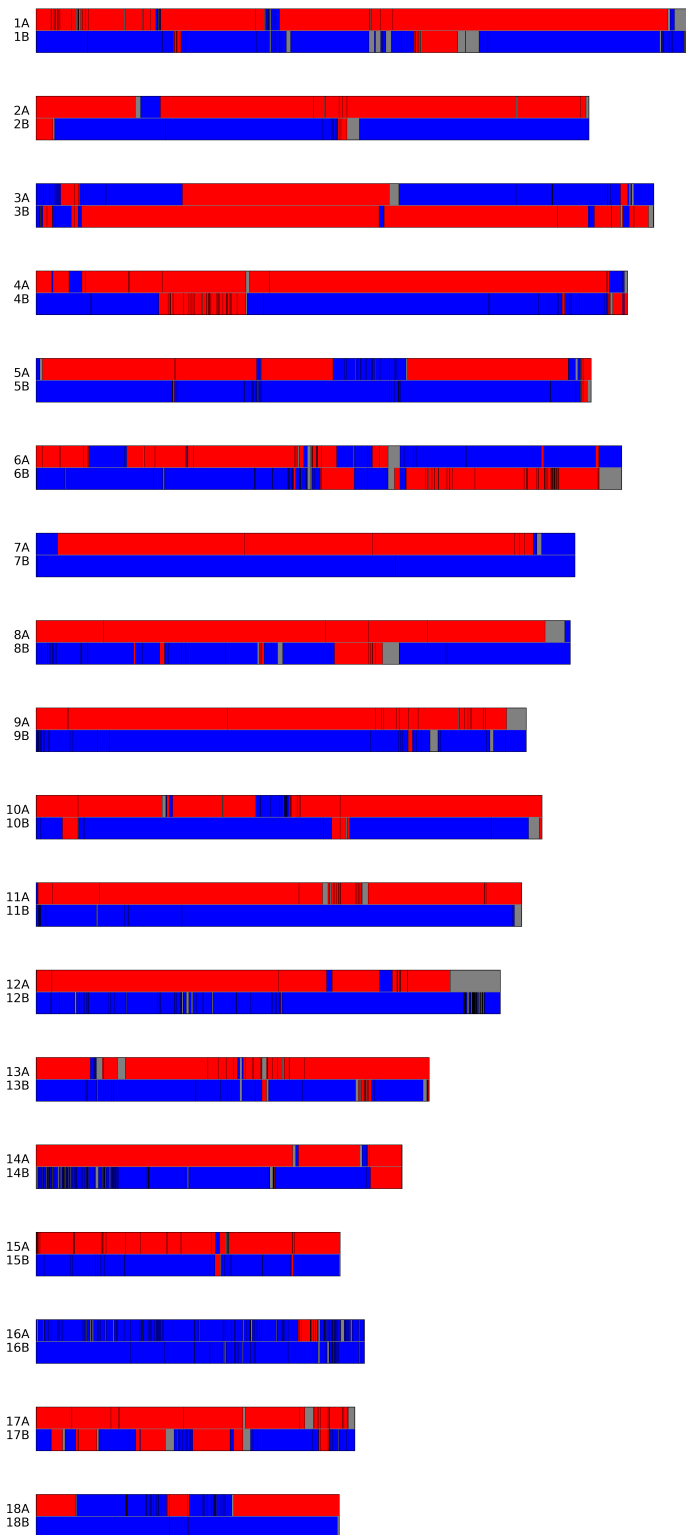


Figure A.5: Chromosome-level phasing results for Hifiasm+GreenHill against Pt76 dataset

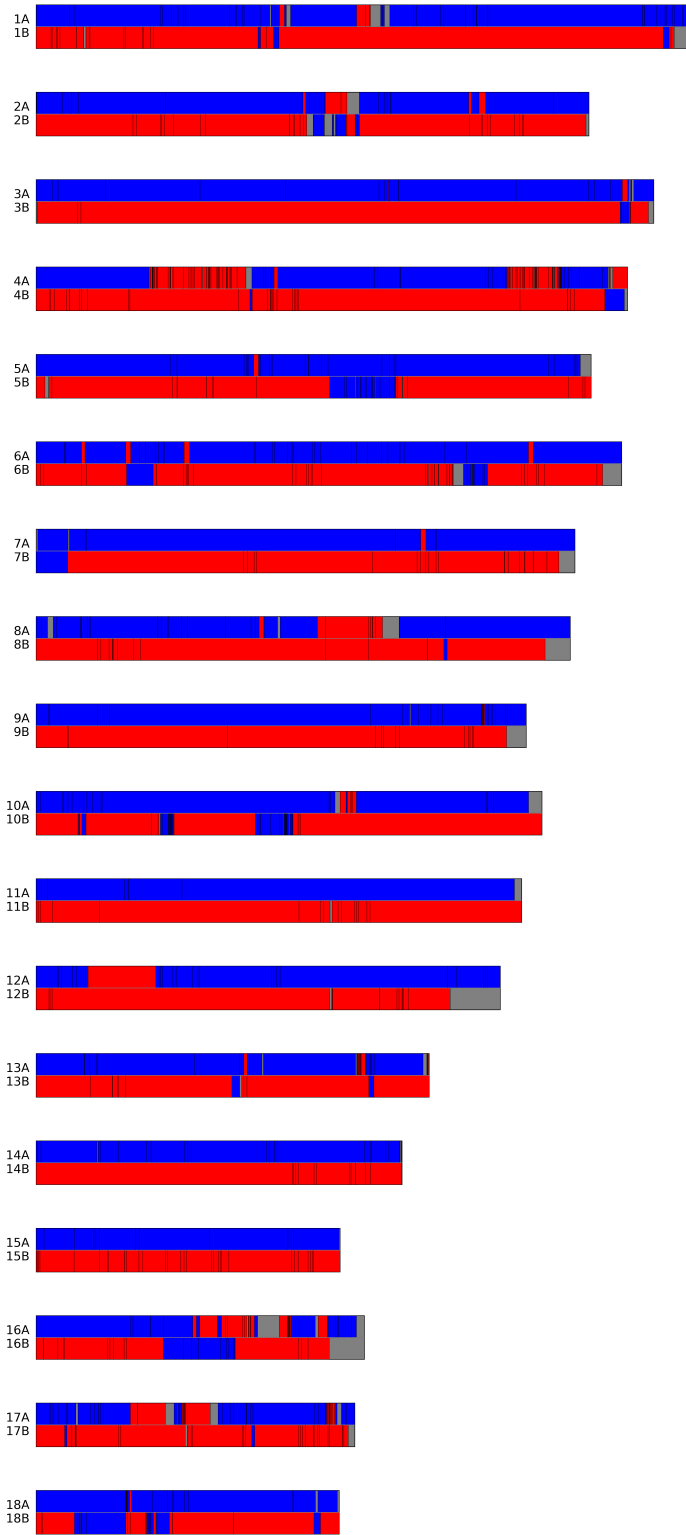


Figure A.6: Chromosome-level phasing results for Hifiasm+Algorithm against Pt76 dataset

Appendix: Chromosome-level Phasing results for Pst104E Dataset

Figure B.1- B.3 contain the chromosome-level phasing results for Pst104E dataset on phasing Verkko assemblies among all chromosomes. Figure B.4- B.6 contain the chromosome-level phasing results for Pst104E dataset on phasing Hifiasm assemblies among all chromosomes.

B Appendix: Chromosome-level Phasing results for Pst104E Dataset

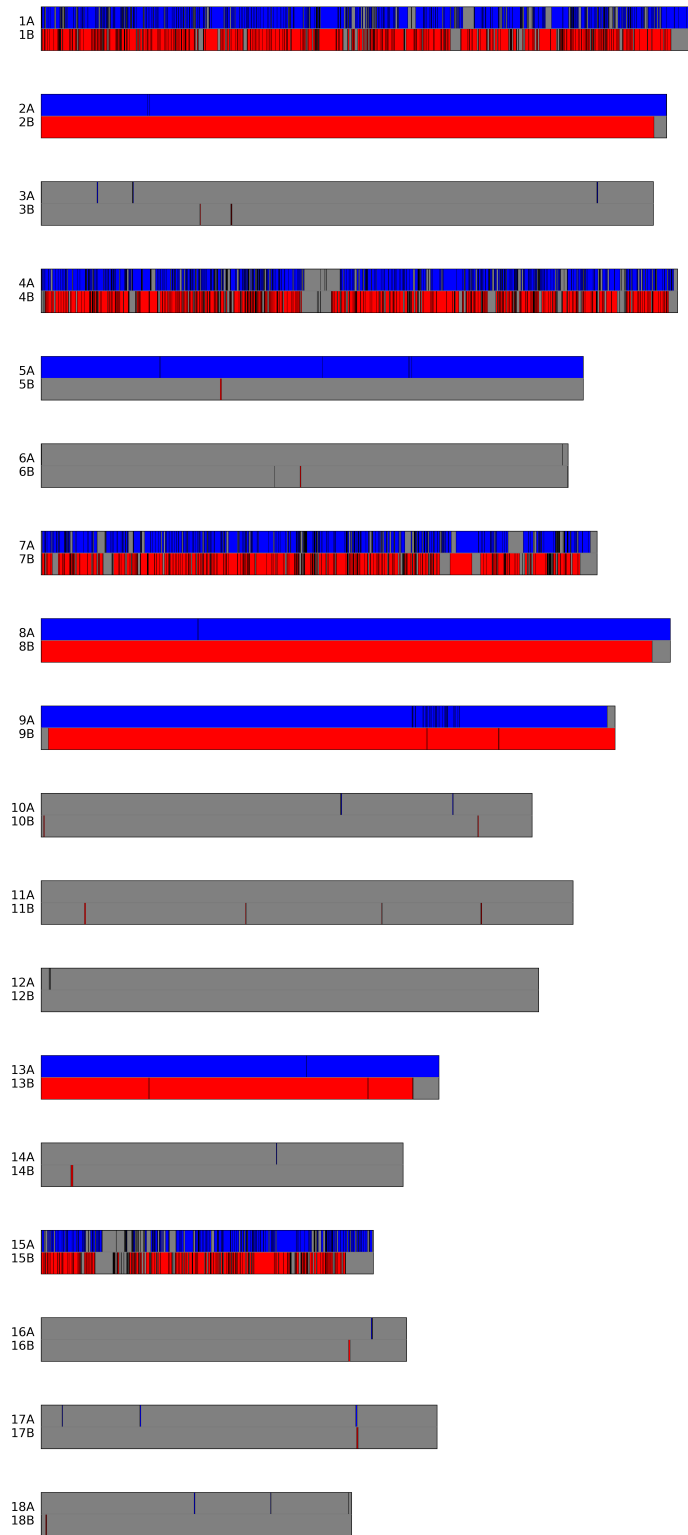


Figure B.1: Chromosome-level phasing results for Verkko (HiC) against Pst104E dataset

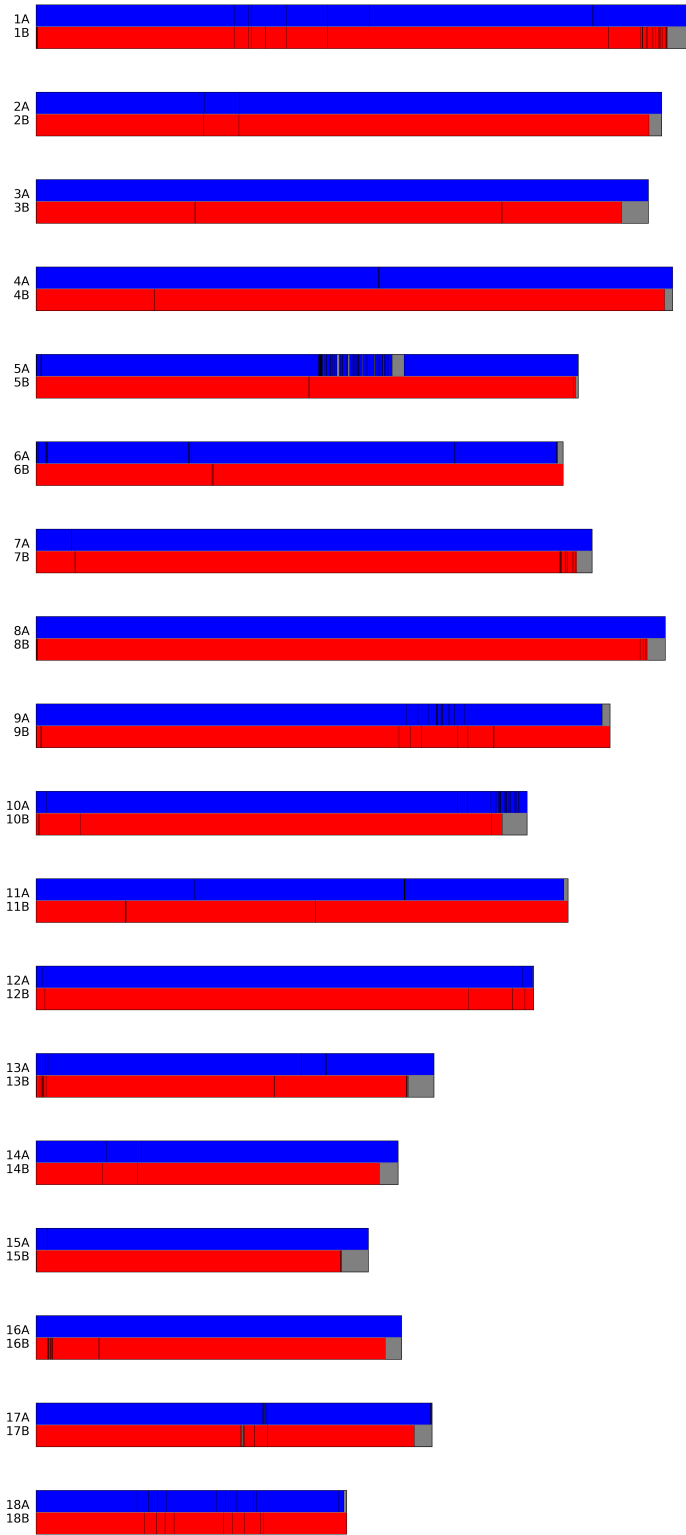


Figure B.2: Chromosome-level phasing results for Verkko+GreenHill against Pst104E dataset

B Appendix: Chromosome-level Phasing results for Pst104E Dataset

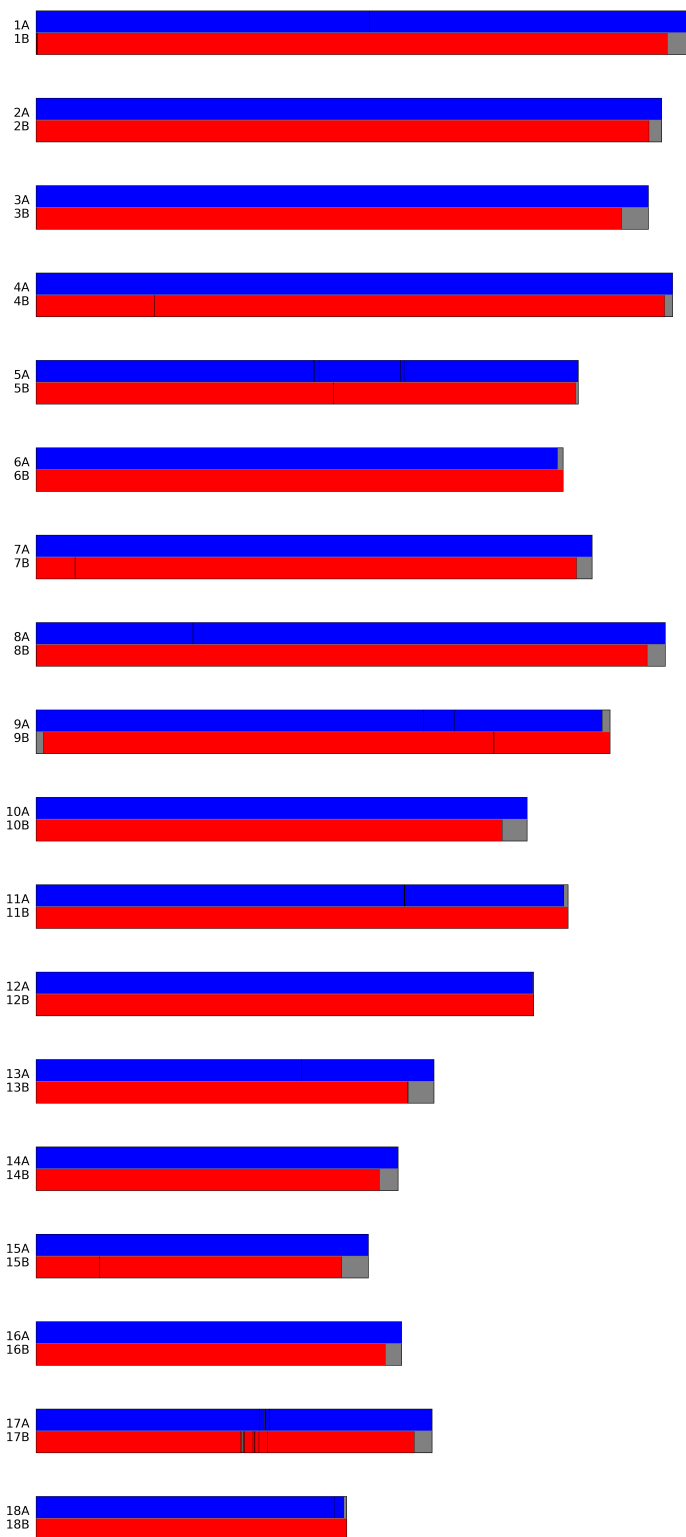


Figure B.3: Chromosome-level phasing results for Verkko+Algorithm against Pst104E dataset

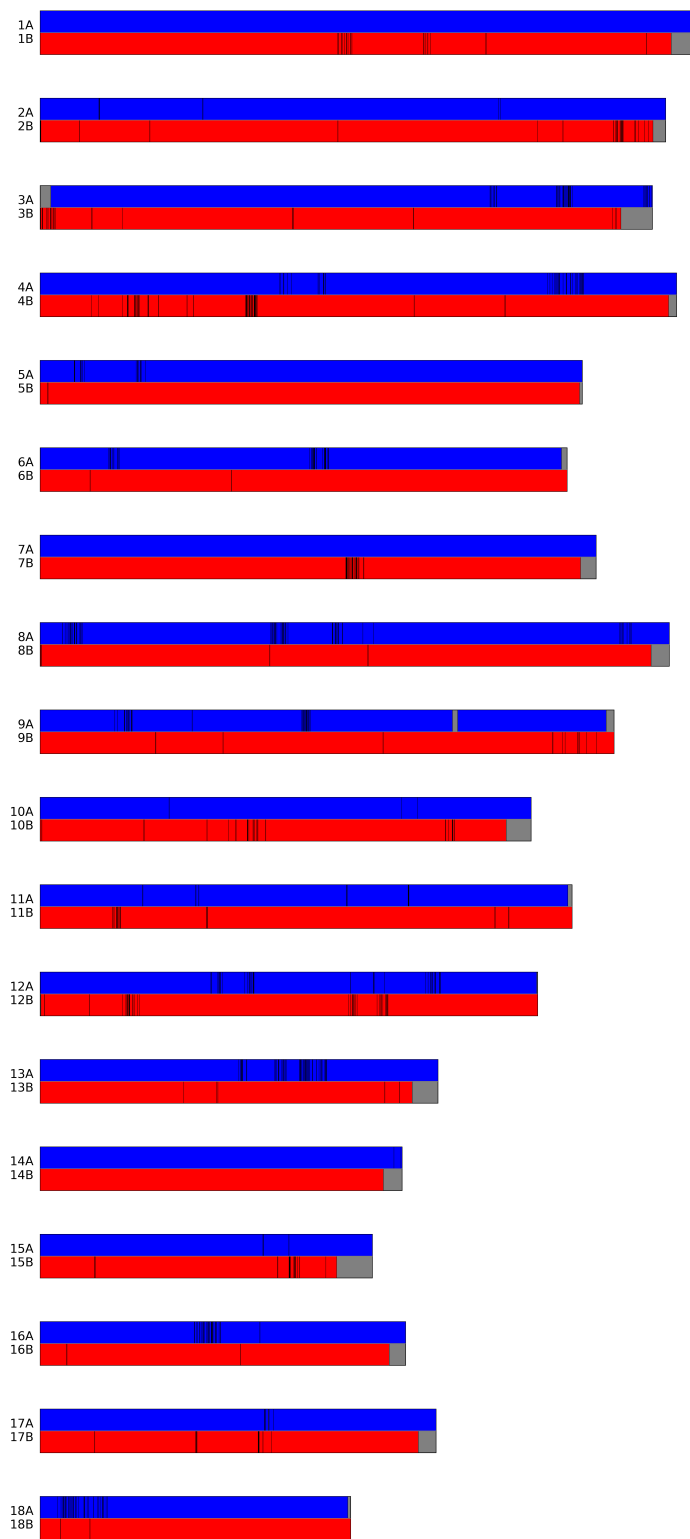


Figure B.4: Chromosome-level phasing results for Hifiasm (HiC) against Pst104E dataset

B Appendix: Chromosome-level Phasing results for Pst104E Dataset

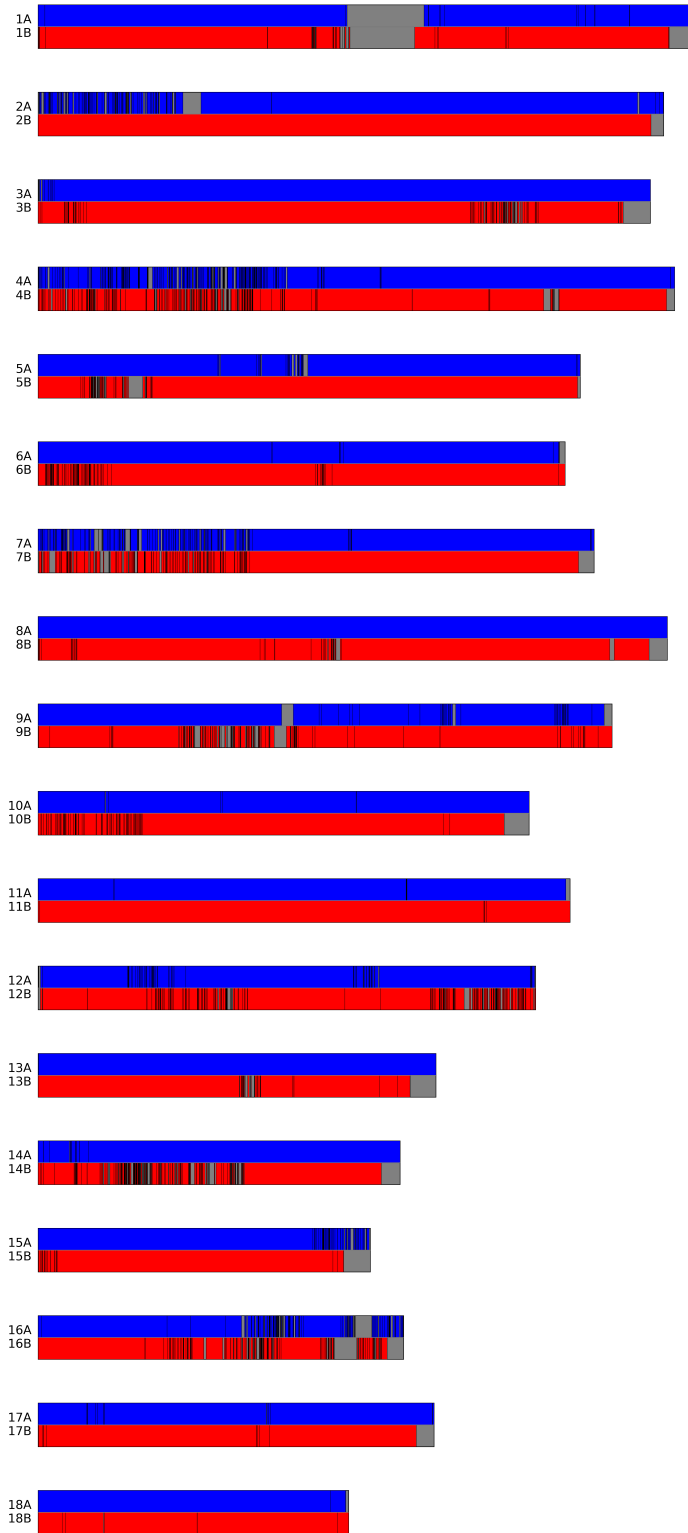


Figure B.5: Chromosome-level phasing results for Hifiasm+GreenHill against Pst104E dataset

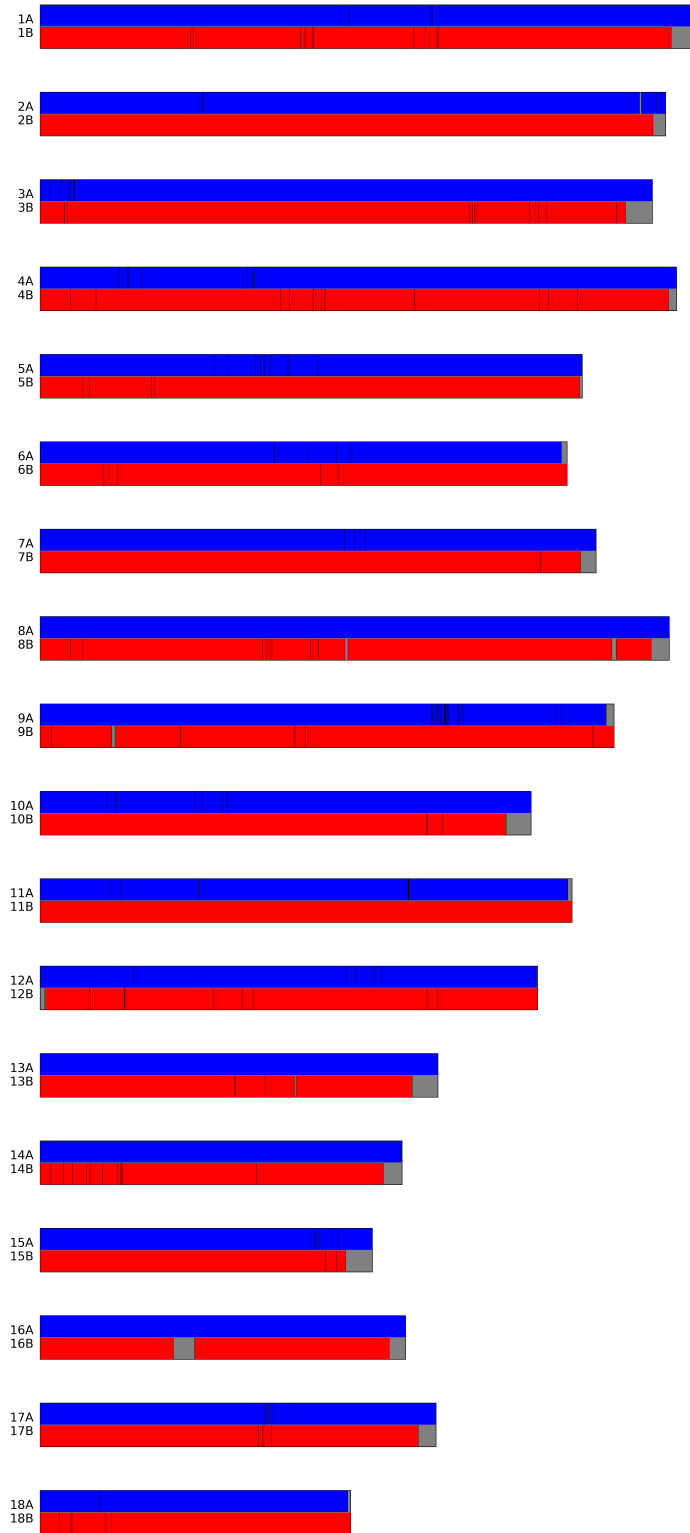


Figure B.6: Chromosome-level phasing results for Hifiasm+Algorithm against Pst104E dataset

Appendix: Command lines

For benchmarking purposes we made use of several existing tools, listed below. All tools under evaluation use default settings unless specified by authors.

Software versions and sources

- Utilities
 - HiC-Pro: v3.1.0, <https://github.com/nservant/HiC-Pro>
 - Kmer2SNP: commit 8670aac, <https://github.com/yanboANU/Kmer2SNP>
 - minimap2: 2.26-r1175, <https://github.com/lh3/minimap2>
 - QUAST: v5.2.0, <https://github.com/ablab/quast>
 - Bowtie2: v2.4.4, <https://github.com/BenLangmead/bowtie2>
 - BWA: 0.7.17-r1198-dirty, <https://github.com/lh3/bwa>
 - MUMMER: v3.23, <https://sourceforge.net/projects/mummer/files/mummer/3.23/>
 - Bandage: v0.9.0, <https://github.com/rrwick/Bandage>
 - GenomeViz: v0.4.3, <https://github.com/moshi4/pyGenomeViz>
- Assembly tools
 - Verkko: bioconda 1.4, <https://github.com/marbl/verkko>
 - Hifiasm: 0.19.6-r595, <https://github.com/chhy123/hifiasm>
 - GreenHill: v1.1.0, <https://github.com/ShunOuchi/GreenHill>

Software commands

C Appendix: Command lines

Variant calling

Kmer2SNP: `bash runkmercalling.sh 21 60 duplex.fastq`

MUMMER:

1. `nucmer --mum --prefix=<prefix> H1.fasta H2.fasta`
2. `delta-filter -1 <prefix>.delta > <prefix>.delta`
3. `show-snps -x 10 -C -T -H -l -q <prefix>.delta > <prefix>.snp`

HiC read alignment

HiC-Pro: `HiC-Pro -i data/ -o hicpro_result/ -c confighicpro.txt`

De novo assembly tools

- Verkko `verkko -d output --hifi duplex.fastq --nano simplex.fastq`
- Verkko (HiC) `verkko -d output --hifi duplex.fastq --nano simplex.fastq --hic1 forward.fastq --hic2 reverse.fastq`
- Hifiasm `hifiasm -o output -t 16 --ul simplex.fastq duplex.fastq`
- Hifiasm (HiC) `hifiasm -o output -t 16 --h1 forward.fastq --h2 reverse.fastq --ul simplex.fastq duplex.fastq`
- GreenHill `greenhill -t 16 -cph assembly.fasta -HIC forward.fastq reverse.fastq -p duplex.fastq -o output`
- Algorithm `python3 src/main.py graph.gfa assembly.fasta hicpro.bed hicpro.matrix kmer2snp.snp kmer2snp.snp.extend output`

Assembly evaluation

QUAST: `python2 quast.py -t 16 --large --skip-unaligned-mis-contigs -r ref.fasta -o output f1.fasta f2.fasta ... fn.fasta`

Bibliography

- ALTSCHUL, S. F.; MADDEN, T. L.; SCHÄFFER, A. A.; ZHANG, J.; ZHANG, Z.; MILLER, W.; AND LIPMAN, D. J., 1997. Gapped blast and psi-blast: a new generation of protein database search programs. *Nucleic acids research*, 25, 17 (1997), 3389–3402. [Cited on page 13.]
- BANKEVICH, A.; BZIKADZE, A. V.; KOLMOGOROV, M.; ANTIPOV, D.; AND PEVZNER, P. A., 2022. Multiplex de bruijn graphs enable genome assembly from long, high-fidelity reads. *Nature biotechnology*, 40, 7 (2022), 1075–1081. [Cited on pages 16 and 23.]
- BANKEVICH, A.; NURK, S.; ANTIPOV, D.; GUREVICH, A. A.; DVORKIN, M.; KULIKOV, A. S.; LESIN, V. M.; NIKOLENKO, S. I.; PHAM, S.; PRJIBELSKI, A. D.; ET AL., 2012. Spades: a new genome assembly algorithm and its applications to single-cell sequencing. *Journal of computational biology*, 19, 5 (2012), 455–477. [Cited on pages 9 and 16.]
- BENTLEY, D. R.; BALASUBRAMANIAN, S.; SWERDLOW, H. P.; SMITH, G. P.; MILTON, J.; BROWN, C. G.; HALL, K. P.; EVERS, D. J.; BARNES, C. L.; BIGNELL, H. R.; ET AL., 2008. Accurate whole human genome sequencing using reversible terminator chemistry. *nature*, 456, 7218 (2008), 53–59. [Cited on page 6.]
- BROWNING, S. R. AND BROWNING, B. L., 2011. Haplotype phasing: existing methods and new developments. *Nature Reviews Genetics*, 12, 10 (2011), 703–714. [Cited on page 18.]
- CHENG, H.; ASRI, M.; LUCAS, J.; KOREN, S.; AND LI, H., 2023. Scalable telomere-to-telomere assembly for diploid and polyploid genomes with double graph. *arXiv preprint arXiv:2306.03399*, (2023). [Cited on pages 17 and 22.]
- CHENG, H.; CONCEPCION, G. T.; FENG, X.; ZHANG, H.; AND LI, H., 2021. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature methods*, 18, 2 (2021), 170–175. [Cited on page 22.]

Bibliography

- CHENG, H.; JARVIS, E. D.; FEDRIGO, O.; KOEPFLI, K.-P.; URBAN, L.; GEMMELL, N. J.; AND LI, H., 2022. Haplotype-resolved assembly of diploid genomes without parental data. *Nature Biotechnology*, 40, 9 (2022), 1332–1335. [Cited on pages 18 and 23.]
- COMPEAU, P. E.; PEVZNER, P. A.; AND TESLER, G., 2011. How to apply de bruijn graphs to genome assembly. *Nature biotechnology*, 29, 11 (2011), 987–991. [Cited on pages 8 and 9.]
- CRAWFORD, D. C. AND NICKERSON, D. A., 2005. Definition and clinical importance of haplotypes. *Annu. Rev. Med.*, 56 (2005), 303–320. [Cited on pages 1 and 57.]
- DE BRUIJN, N. G., 1946. A combinatorial problem. *Proceedings of the Section of Sciences of the Koninklijke Nederlandse Akademie van Wetenschappen te Amsterdam*, 49, 7 (1946), 758–764. [Cited on pages 7, 8, and 9.]
- DEAMER, D.; AKESON, M.; AND BRANTON, D., 2016. Three decades of nanopore sequencing. *Nature biotechnology*, 34, 5 (2016), 518–524. [Cited on page 6.]
- DELCHER, A. L.; SALZBERG, S. L.; AND PHILLIPPY, A. M., 2003. Using mummer to identify similar regions in large sequence sets. *Current protocols in bioinformatics*, , 1 (2003), 10–3. [Cited on page 44.]
- DIJKSTRA, E. W., 2022. A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*, 287–290. [Cited on page 39.]
- DOSTIE, J.; RICHMOND, T. A.; ARNAOUT, R. A.; SELZER, R. R.; LEE, W. L.; HONAN, T. A.; RUBIO, E. D.; KRUMM, A.; LAMB, J.; NUSBAUM, C.; ET AL., 2006. Chromosome conformation capture carbon copy (5c): a massively parallel solution for mapping interactions between genomic elements. *Genome research*, 16, 10 (2006), 1299–1309. [Cited on page 7.]
- DUAN, H.; JONES, A. W.; HEWITT, T.; MACKENZIE, A.; HU, Y.; SHARP, A.; LEWIS, D.; MAGO, R.; UPADHYAYA, N. M.; RATHJEN, J. P.; ET AL., 2022. Physical separation of haplotypes in dikaryons allows benchmarking of phasing accuracy in nanopore and hifi assemblies with hi-c data. *Genome biology*, 23, 1 (2022), 84. [Cited on pages 4 and 43.]
- EDGE, P.; BAFNA, V.; AND BANSAL, V., 2017. Hapcut2: robust and accurate haplotype assembly for diverse sequencing technologies. *Genome research*, 27, 5 (2017), 801–812. [Cited on page 24.]
- EDMONDS, J., 1965. Paths, trees, and flowers. *Canadian Journal of mathematics*, 17 (1965), 449–467. [Cited on page 32.]
- EID, J.; FEHR, A.; GRAY, J.; LUONG, K.; LYLE, J.; OTTO, G.; PELUSO, P.; RANK, D.; BAYBAYAN, P.; BETTMAN, B.; ET AL., 2009. Real-time dna sequencing from single polymerase molecules. *Science*, 323, 5910 (2009), 133–138. [Cited on page 6.]

- EKIM, B.; BERGER, B.; AND CHIKHI, R., 2021. Minimizer-space de bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer. *Cell systems*, 12, 10 (2021), 958–968. [Cited on page 16.]
- EULER, L., 1956. The seven bridges of königsberg. *The world of mathematics*, 1 (1956), 573–580. [Cited on page 7.]
- GHAJ, R.; HAIN, T.; AND CHAKRABORTY, T., 2004. Genomeviz: visualizing microbial genomes. *BMC bioinformatics*, 5, 1 (2004), 1–6. [Cited on page 48.]
- GOOD, I. J., 1946. Normal recurring decimals. *Journal of the London Mathematical Society*, 1, 3 (1946), 167–169. [Cited on pages 7 and 8.]
- GUREVICH, A.; SAVELIEV, V.; VYAHHI, N.; AND TESLER, G., 2013. Quast: quality assessment tool for genome assemblies. *Bioinformatics*, 29, 8 (2013), 1072–1075. [Cited on page 44.]
- HARRIS, T. D.; BUZBY, P. R.; BABCOCK, H.; BEER, E.; BOWERS, J.; BRASLAVSKY, I.; CAUSEY, M.; COLONELL, J.; DiMEO, J.; EFCAVITCH, J. W.; ET AL., 2008. Single-molecule dna sequencing of a viral genome. *Science*, 320, 5872 (2008), 106–109. [Cited on page 6.]
- HEATHER, J. M. AND CHAIN, B., 2016. The sequence of sequencers: The history of sequencing dna. *Genomics*, 107, 1 (2016), 1–8. [Cited on page 5.]
- JOHNSON, G. C.; ESPOSITO, L.; BARRATT, B. J.; SMITH, A. N.; HEWARD, J.; DI GENOVA, G.; UEDA, H.; CORDELL, H. J.; EAVES, I. A.; DUDBRIDGE, F.; ET AL., 2001. Haplotype tagging for the identification of common disease genes. *Nature genetics*, 29, 2 (2001), 233–237. [Cited on page 1.]
- KENT, W. J., 2002. Blat—the blast-like alignment tool. *Genome research*, 12, 4 (2002), 656–664. [Cited on page 13.]
- KERNIGHAN, B. W. AND LIN, S., 1970. An efficient heuristic procedure for partitioning graphs. *The Bell system technical journal*, 49, 2 (1970), 291–307. [Cited on pages 34 and 36.]
- KOLMOGOROV, M.; YUAN, J.; LIN, Y.; AND PEVZNER, P. A., 2019. Assembly of long, error-prone reads using repeat graphs. *Nature biotechnology*, 37, 5 (2019), 540–546. [Cited on page 21.]
- KOREN, S.; RHIE, A.; WALENZ, B. P.; DILTHEY, A. T.; BICKHART, D. M.; KINGAN, S. B.; HIENDLEDER, S.; WILLIAMS, J. L.; SMITH, T. P.; AND PHILLIPPY, A. M., 2018. De novo assembly of haplotype-resolved genomes with trio binning. *Nature biotechnology*, 36, 12 (2018), 1174–1182. [Cited on pages 1 and 18.]

Bibliography

- KOREN, S.; WALENZ, B. P.; BERLIN, K.; MILLER, J. R.; BERGMAN, N. H.; AND PHILLIPPY, A. M., 2017. Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation. *Genome research*, 27, 5 (2017), 722–736. [Cited on page 21.]
- KRONENBERG, Z. N.; RHIE, A.; KOREN, S.; CONCEPCION, G. T.; PELUSO, P.; MUNSON, K. M.; PORUBSKY, D.; KUHN, K.; MUELLER, K. A.; LOW, W. Y.; ET AL., 2021. Extended haplotype-phasing of long-read de novo genome assemblies using hi-c. *Nature communications*, 12, 1 (2021), 1935. [Cited on page 1.]
- LAJOIE, B. R.; DEKKER, J.; AND KAPLAN, N., 2015. The hitchhiker’s guide to hi-c analysis: practical guidelines. *Methods*, 72 (2015), 65–75. [Cited on page 6.]
- LANGMEAD, B. AND SALZBERG, S. L., 2012. Fast gapped-read alignment with bowtie 2. *Nature methods*, 9, 4 (2012), 357–359. [Cited on page 23.]
- LI, H., 2013. Aligning sequence reads, clone sequences and assembly contigs with bwa-mem. *arXiv preprint arXiv:1303.3997*, (2013). [Cited on pages 23 and 24.]
- LI, H., 2016. Minimap and miniasm: fast mapping and de novo assembly for noisy long sequences. *Bioinformatics*, 32, 14 (2016), 2103–2110. [Cited on pages 11 and 13.]
- LI, H., 2018. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34, 18 (2018), 3094–3100. [Cited on page 25.]
- LI, H. AND DURBIN, R., 2023. Genome assembly in the telomere-to-telomere era. *ArXiv*, (2023). [Cited on pages 10, 15, 17, and 57.]
- LI, H.; HANDSAKER, B.; WYSOKER, A.; FENNELL, T.; RUAN, J.; HOMER, N.; MARTH, G.; ABECASIS, G.; DURBIN, R.; AND SUBGROUP, . G. P. D. P., 2009. The sequence alignment/map format and samtools. *bioinformatics*, 25, 16 (2009), 2078–2079. [Cited on page 4.]
- LI, Y.; PATEL, H.; AND LIN, Y., 2020. Kmer2snp: reference-free snp calling from raw reads based on matching. In *2020 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 208–212. IEEE. [Cited on pages 26 and 31.]
- LIEBERMAN-AIDEN, E.; VAN BERKUM, N. L.; WILLIAMS, L.; IMAKAEV, M.; RAGOCZY, T.; TELLING, A.; AMIT, I.; LAJOIE, B. R.; SABO, P. J.; DORSCHNER, M. O.; ET AL., 2009. Comprehensive mapping of long-range interactions reveals folding principles of the human genome. *science*, 326, 5950 (2009), 289–293. [Cited on page 7.]
- LIN, Y.; YUAN, J.; KOLMOGOROV, M.; SHEN, M. W.; CHAISSON, M.; AND PEVZNER, P. A., 2016. Assembly of long error-prone reads using de bruijn graphs. *Proceedings of the National Academy of Sciences*, 113, 52 (2016), E8396–E8405. [Cited on page 21.]

- MILLER, J. R.; DELCHER, A. L.; KOREN, S.; VENTER, E.; WALENZ, B. P.; BROWNLEY, A.; JOHNSON, J.; LI, K.; MOBARRY, C.; AND SUTTON, G., 2008. Aggressive assembly of pyrosequencing reads with mates. *Bioinformatics*, 24, 24 (2008), 2818–2824. [Cited on page 21.]
- MYERS, E. W., 1995. Toward simplifying and accurately formulating fragment assembly. *Journal of Computational Biology*, 2, 2 (1995), 275–290. [Cited on pages 7 and 10.]
- MYERS, E. W., 2005. The fragment assembly string graph. *Bioinformatics*, 21, suppl_2 (2005), ii79–ii85. [Cited on pages 7 and 10.]
- NEEDLEMAN, S. B. AND WUNSCH, C. D., 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of molecular biology*, 48, 3 (1970), 443–453. [Cited on page 11.]
- NURK, S.; WALENZ, B. P.; RHIE, A.; VOLLGER, M. R.; LOGSDON, G. A.; GROTHE, R.; MIGA, K. H.; EICHLER, E. E.; PHILLIPPY, A. M.; AND KOREN, S., 2020. Hicnu: accurate assembly of segmental duplications, satellites, and allelic variants from high-fidelity long reads. *Genome research*, 30, 9 (2020), 1291–1305. [Cited on page 22.]
- OUCHI, S.; KAJITANI, R.; AND ITOH, T., 2023. Greenhill: a de novo chromosome-level scaffolding and phasing tool using hi-c. *Genome Biology*, 24, 1 (2023), 162. [Cited on pages 23 and 25.]
- PEVZNER, P. A.; TANG, H.; AND WATERMAN, M. S., 2001. An eulerian path approach to dna fragment assembly. *Proceedings of the national academy of sciences*, 98, 17 (2001), 9748–9753. [Cited on pages 7 and 9.]
- RAUTIAINEN, M. AND MARSCHALL, T., 2021. Mbg: Minimizer-based sparse de bruijn graph construction. *Bioinformatics*, 37, 16 (2021), 2476–2478. [Cited on pages 16 and 22.]
- RAUTIAINEN, M.; NURK, S.; WALENZ, B. P.; LOGSDON, G. A.; PORUBSKY, D.; RHIE, A.; EICHLER, E. E.; PHILLIPPY, A. M.; AND KOREN, S., 2023. Telomere-to-telomere assembly of diploid chromosomes with verkko. *Nature Biotechnology*, (2023), 1–9. [Cited on pages 22 and 23.]
- RHIE, A.; WALENZ, B. P.; KOREN, S.; AND PHILLIPPY, A. M., 2020. Merquy: reference-free quality, completeness, and phasing assessment for genome assemblies. *Genome biology*, 21, 1 (2020), 1–27. [Cited on page 44.]
- RIZK, G.; LAVENIER, D.; AND CHIKHI, R., 2013. Dsk: k-mer counting with very low memory usage. *Bioinformatics*, 29, 5 (2013), 652–653. [Cited on page 26.]
- SANGER, F. AND COULSON, A. R., 1975. A rapid method for determining sequences in dna by primed synthesis with dna polymerase. *Journal of molecular biology*, 94, 3 (1975), 441–448. [Cited on page 6.]

Bibliography

- SANGER, F.; NICKLEN, S.; AND COULSON, A. R., 1977. Dna sequencing with chain-terminating inhibitors. *Proceedings of the national academy of sciences*, 74, 12 (1977), 5463–5467. [Cited on page 6.]
- SELVARAJ, S.; R DIXON, J.; BANSAL, V.; AND REN, B., 2013. Whole-genome haplotype reconstruction using proximity-ligation and shotgun sequencing. *Nature biotechnology*, 31, 12 (2013), 1111–1118. [Cited on page 18.]
- SERVANT, N.; VAROQUAUX, N.; LAJOIE, B. R.; VIARA, E.; CHEN, C.-J.; VERT, J.-P.; HEARD, E.; DEKKER, J.; AND BARILLOT, E., 2015. Hic-pro: an optimized and flexible pipeline for hi-c data processing. *Genome biology*, 16, 1 (2015), 1–11. [Cited on page 33.]
- SIMONIS, M.; KLOUS, P.; SPLINTER, E.; MOSHKIN, Y.; WILLEMSSEN, R.; DE WIT, E.; VAN STEENSEL, B.; AND DE LAAT, W., 2006. Nuclear organization of active and inactive chromatin domains uncovered by chromosome conformation capture–on-chip (4c). *Nature genetics*, 38, 11 (2006), 1348–1354. [Cited on page 7.]
- SMITH, T. F.; WATERMAN, M. S.; ET AL., 1981. Identification of common molecular subsequences. *Journal of molecular biology*, 147, 1 (1981), 195–197. [Cited on page 11.]
- SUN, H.; DING, J.; PIEDNOËL, M.; AND SCHNEEBERGER, K., 2018. findgse: estimating genome size variation within human and arabidopsis using k-mer frequencies. *Bioinformatics*, 34, 4 (2018), 550–557. [Cited on page 26.]
- TAKAHASHI, H. AND MATSUYAMA, A., 1980. An approximate solution for the steiner problem in graphs. *Mathematica Japonica*, 24, 6 (1980), 573–577. [Cited on page 39.]
- VENTER, J. C.; ADAMS, M. D.; MYERS, E. W.; LI, P. W.; MURAL, R. J.; SUTTON, G. G.; SMITH, H. O.; YANDELL, M.; EVANS, C. A.; HOLT, R. A.; ET AL., 2001. The sequence of the human genome. *science*, 291, 5507 (2001), 1304–1351. [Cited on page 6.]
- WICK, R. R.; SCHULTZ, M. B.; ZOBEL, J.; AND HOLT, K. E., 2015. Bandage: interactive visualization of de novo genome assemblies. *Bioinformatics*, 31, 20 (2015), 3350–3352. [Cited on page 48.]